```
if not SUPPORTS_FLOATING_WINDOWS then
  logMsg("imgui not supported by your FlyWithLua version")
  return
end

require("bit")
require("graphics")

local socket_ok, socket = pcall(require, "socket")
local http_ok, http     = pcall(require, "socket.http")

if not socket_ok or not http_ok then
  logMsg("ERROR - socket.lua or socket.http not found")
end

local ffi = require ("ffi")
local XPLMlib = ""

if SYSTEM == "IBM" then
  if SYSTEM_ARCHITECTURE == 64 then
    XPLMlib = "XPLM_64"
  else
    XPLMlib = "XPLM"
  end
elseif SYSTEM == "LIN" then
  if SYSTEM_ARCHITECTURE == 64 then
    XPLMlib = "Resources/plugins/XPLM_64.so"
  else
    XPLMlib = "Resources/plugins/XPLM.so"
  end
elseif SYSTEM == "APL" then
  XPLMlib = "Resources/plugins/XPLM.framework/XPLM"
else
  return
end

local XPLM = ffi.load(XPLMlib)

local cdefs = [[
enum {
```

```
  xplm_ControlCameraUntilViewChanges       = 1
  ,xplm_ControlCameraForever               = 2
};

typedef struct {
  int                    structSize;
  float                  x;
  float                  y;
  float                  z;
  float                  pitch;
  float                  heading;
  float                  roll;
} XPLMDrawInfo_t;

typedef struct {
  int                    structSize;
  float                  locationX;
  float                  locationY;
  float                  locationZ;
  float                  normalX;
  float                  normalY;
  float                  normalZ;
  float                  velocityX;
  float                  velocityY;
  float                  velocityZ;
  int                    is_wet;
} XPLMProbeInfo_t;

typedef struct {
  float                  x;
  float                  y;
  float                  z;
  float                  pitch;
  float                  heading;
  float                  roll;
  float                  zoom;
} XPLMCameraPosition_t;

typedef void *inRefcon;
typedef void *XPLMDataRef;
```

```
typedef void *XPLMObjectRef;
typedef void *XPLMInstanceRef;
typedef void *XPLMProbeRef;
typedef int XPLMProbeType;
typedef int XPLMProbeResult;
typedef int XPLMCameraControlDuration;

typedef int (* XPLMCameraControl_f)(XPLMCameraPosition_t *outCameraPosition,
int inIsLosingControl,
void *inRefcon);

typedef void (*XPLMObjectLoaded_f)(XPLMObjectRef inObject, void *inRefcon);

typedef int (*XPLMGetDatai_f)(void *inRefcon);
typedef void (*XPLMSetDatai_f)(void *inRefcon, int inValue);
typedef float (*XPLMGetDataf_f)(void *inRefcon);
typedef void (*XPLMSetDataf_f)(void *inRefcon, float inValue);
typedef double (*XPLMGetDatad_f)(void *inRefcon);
typedef void (*XPLMSetDatad_f)(void *inRefcon, double inValue);
typedef int (*XPLMGetDatavi_f)(void *inRefcon, int *outValues, int inOffset, int inMax);
typedef void (*XPLMSetDatavi_f)(void *inRefcon, int *inValues, int inOffset, int inCount);
typedef int (*XPLMGetDatavf_f)(void *inRefcon, float *outValues, int inOffset, int inMax);
typedef void (*XPLMSetDatavf_f)(void *inRefcon, float *inValues, int inOffset, int inCount);
typedef int (*XPLMGetDatab_f)(void *inRefcon, void *outValue, int inOffset, int inMaxLength);
typedef void (*XPLMSetDatab_f)(void *inRefcon, void *inValue, int inOffset, int inLength);

XPLMDataRef XPLMRegisterDataAccessor(
const char *        inDataName,
int                 inDataType,
int                 inIsWritable,
XPLMGetDatai_f      inReadInt,
XPLMSetDatai_f      inWriteInt,
XPLMGetDataf_f      inReadFloat,
XPLMSetDataf_f      inWriteFloat,
XPLMGetDatad_f      inReadDouble,
XPLMSetDatad_f      inWriteDouble,
XPLMGetDatavi_f     inReadIntArray,
XPLMSetDatavi_f     inWriteIntArray,
XPLMGetDatavf_f     inReadFloatArray,
XPLMSetDatavf_f     inWriteFloatArray,
```

```
XPLMGetDatab_f       inReadData,
XPLMSetDatab_f       inWriteData,
void *               inReadRefcon,
void *               inWriteRefcon);

XPLMObjectRef XPLMLoadObject( const char *inPath);
void XPLMLoadObjectAsync( const char * inPath, XPLMObjectLoaded_f inCallback, void *inRefcon);
XPLMInstanceRef XPLMCreateInstance(XPLMObjectRef obj, const char **datarefs);
void XPLMInstanceSetPosition(XPLMInstanceRef instance, const XPLMDrawInfo_t *new_position, const float
*data);
XPLMProbeRef XPLMCreateProbe(XPLMProbeType inProbeType);
XPLMProbeResult XPLMProbeTerrainXYZ( XPLMProbeRef inProbe, float inX, float inY, float inZ, XPLMProbeInfo_t
*outInfo);
void XPLMUnregisterDataAccessor(XPLMDataRef inDataRef);
void XPLMDestroyInstance(XPLMInstanceRef instance);
void XPLMUnloadObject(XPLMObjectRef inObject);
void XPLMDestroyProbe(XPLMProbeRef inProbe);
void XPLMControlCamera(XPLMCameraControlDuration inHowLong, XPLMCameraControl_f  inControlFunc, void
*inRefcon);
void XPLMDontControlCamera(void);
int XPLMIsCameraBeingControlled(XPLMCameraControlDuration *outCameraControlDuration);
void XPLMReadCameraPosition(XPLMCameraPosition_t *outCameraPosition);
void XPLMWorldToLocal( double inLatitude, double inLongitude, double inAltitude, double *outX, double *outY,
double *outZ);
void XPLMLocalToWorld( double inX, double inY, double inZ, double *outLatitude, double *outLongitude, double
*outAltitude);
void XPLMGetSystemPath(char * outSystemPath);
]]

ffi.cdef(cdefs)

local char_str = ffi.new("char[256]")
local datarefs_addr = ffi.new("const char**")
local dataref_name = ffi.new("char[150]")

local dataref_register1 = ffi.new("XPLMDataRef")
local dataref_register2 = ffi.new("XPLMDataRef")
local dataref_register3 = ffi.new("XPLMDataRef")
local dataref_register4 = ffi.new("XPLMDataRef")
local dataref_register5 = ffi.new("XPLMDataRef")
```

```
local dataref_register6 = ffi.new("XPLMDataRef")
local dataref_array = ffi.new("const char*[7]")

local dataref_register7 = ffi.new("XPLMDataRef")
local dataref_array2 = ffi.new("const char*[2]")

local objref = ffi.new("XPLMObjectRef")
local signboardref = ffi.new("XPLMObjectRef")
local pathref = ffi.new("XPLMObjectRef")
local rampstartref = ffi.new("XPLMObjectRef")
local proberef = ffi.new("XPLMProbeRef")

local obj_instance = ffi.new("XPLMInstanceRef[1]")
local signboard_instance = ffi.new("XPLMInstanceRef[1]")
local path_instance = ffi.new("XPLMInstanceRef[100]")
local rampstart_instance = ffi.new("XPLMInstanceRef[1]")

local objpos_addr =  ffi.new("const XPLMDrawInfo_t*")
local objpos_value = ffi.new("XPLMDrawInfo_t[1]")
local float_addr = ffi.new("const float*")
local float_value = ffi.new("float[1]")
local dataref_float_addr = ffi.new("const float*")
local dataref_float_value = ffi.new("float[7]")
local dataref_float_value2 = ffi.new("float[2]")
local probeinfo_addr =  ffi.new("XPLMProbeInfo_t*")
local probeinfo_value = ffi.new("XPLMProbeInfo_t[1]")
local probetype = ffi.new("int[1]");
local x1_value = ffi.new("double[1]")
local y1_value = ffi.new("double[1]")
local z1_value = ffi.new("double[1]")

local syspath = ""
local BUFSIZE = 102400

local flightstart = 0

local followme_wnd = nil
local holder_wnd = nil

local screen_width = 1920
```

```
local Holder_len = 30
local prev_mouse_Y = 0
local holder_drag = 0
local toggle_window = false
local window_is_open = false
local text_was_chg = false

local prepare_show_objects = false
local prepare_kill_objects = false
local kill_is_manual = false

local init_load = 0
local window_first_access = false
local FM_car_active = false

local depart_arrive = 0
local depart_gate, arrival_gate, depart_runway, gatetext = 0, 0, "", ""

local curr_ICAO, curr_ICAO_Name = "", ""
local t_runway, t_runway_node, t_gate, t_taxinode, t_segment = {}, {}, {}, {}, {}

local t_deleted_runway = {}

local t_stack = {}
local t_possible_route = {}
local t_node = {}
local t_suitable_gates = {}

local dist_prev_steerpt = 9999
local dist_min_steerpt = 9999
local target_filename = ""
local target_index = 0

local Err_Msg = {}
Err_Msg[1] = {}
Err_Msg[2] = {}
Err_Msg[3] = {}

local ac_types = {"0 - Fighter",
"1 - SUPER HEAVY JET - A-380, C-5, 747",
```

```
"2 - HEAVY JET - A-340, A-330, 777",
"3 - LARGE JET - A-320, 737, 757",
"4 - LARGE PROP - A-400, Hercules, C130",
"5 - MEDIUM JET - Regional Jets CRJ, ERJ",
"6 - MEDIUM PROP - Regional Prop planes Dash8-100",
"7 - LIGHT JET - Learjet, Gulfstream, Fighter",
"8 - LIGHT PROP - GA prop planes"
}

local Win_Y = 400
local Aircraft_Type = "8"
local get_from_SimBrief = false
local show_path = false
local show_rampstart = false
local impose_restriction_chk = true
local random_gate = false
local vol = 5
local speed_limiter = false

local car_type_fmcar = "Auto"

local simbrief_id        = ""
local sb_origin_icao     = ""
local sb_origin_name     = ""
local sb_dest_icao       = ""
local sb_dest_name       = ""
local sb_runway_takeoff  = ""
local sb_runway_landing  = ""
local sb_fetch_status    = ""
local sb_airport_mismatch = false

dataref("viewext", "sim/graphics/view/view_is_external")
dataref("camera_z_position", "sim/graphics/view/pilots_head_z")
dataref("fm_gear1_gnd", "sim/flightmodel2/gear/on_ground", "readonly", 0)
dataref("fm_gear2_gnd", "sim/flightmodel2/gear/on_ground", "readonly", 1)

dataref("fm_new_flight", "sim/time/total_flight_time_sec")
dataref("fm_run_time", "sim/time/total_running_time_sec")
dataref("fm_sim_time", "sim/operation/misc/frame_rate_period")
dataref("fm_replay", "sim/time/is_in_replay")
```

```
dataref("fm_plane_x", "sim/flightmodel/position/local_x")
dataref("fm_plane_y", "sim/flightmodel/position/local_y")
dataref("fm_plane_z", "sim/flightmodel/position/local_z")

dataref("fm_gnd_spd", "sim/flightmodel/position/groundspeed", "readonly")
dataref("fm_plane_head", "sim/flightmodel/position/psi", "readonly")

local fm_taxi_light = 0
local fm_beacon_light = 0

if (string.lower(PLANE_AUTHOR) == "gliding kiwi") or
(string.lower(PLANE_AUTHOR) == "glidingkiwi") then
dataref("fm_taxi_light", "AirbusFBW/OHPLightSwitches", "readonly", 3)
dataref("fm_beacon_light", "AirbusFBW/OHPLightSwitches", "readonly", 7)
else
dataref("fm_taxi_light", "sim/cockpit/electrical/taxi_light_on", "readonly")
dataref("fm_beacon_light", "sim/cockpit/electrical/strobe_lights_on", "readonly")
end

local snd_arrived = load_WAV_file(SCRIPT_DIRECTORY .. "follow_me/sounds/arrived.wav")
local snd_followme = load_WAV_file(SCRIPT_DIRECTORY .. "follow_me/sounds/followme.wav")
local snd_safeflight_bye = load_WAV_file(SCRIPT_DIRECTORY .. "follow_me/sounds/safeflight_goodbye.wav")
local snd_welcome = load_WAV_file(SCRIPT_DIRECTORY .. "follow_me/sounds/welcome_followme.wav")
local snd_welcome_bye = load_WAV_file(SCRIPT_DIRECTORY ..
"follow_me/sounds/welcomeagain_goodbye.wav")

local path_is_shown = false
local rampstart_chg = false
local world_alt = 0

local gravity = 9.81
local reaction_time = 0.68
local cof = 0.35
local deccel_max = -8.9
local deccel_avg = -4
local accel_avg = 2

local car_weight = 1445
local tire_diameter = 0.79
local min_turn_radius = 10.8
```

```
local width_btw_midtire = 1.86
local car_front_to_back_wheel = 3.2
local car_rear_wheel_to_ref = 1.60
local time_to_100 = 3
local speed_max = 88.88
local place_above_the_car = 1.43
local place_Z_of_car = 0

local min_dist_from_plane = 30
local avg_dist_from_plane = 65
local max_dist_from_plane = 80

local accel_max = 27.7778 / time_to_100
local max_steering = math.deg ( math.atan ( car_front_to_back_wheel / min_turn_radius ) )
local min_rot_radius = min_turn_radius + ( width_btw_midtire / 2 )
local car_default_speed = speed_max

local i = 0
local curr_node = 1
local tire_rotate = 0
local steering = 0
local car_sign = 0

local elapsed_time = 0
local car_speed = 0
local car_accel = 0
local car_x, car_y, car_z = 0, 0, 0
local remaining_dist_leg = 0
local turning_is_active = 0
local prev_taxi_light = 0
local prev_beacon_light = 0
local ground_time = 0
local taxiway_network = ""
local play_time = 0
local play_text = ""

function check_SimBrief()

sb_fetch_status = ""
sb_airport_mismatch = false
```

```
if simbrief_id == "" then
  sb_fetch_status = "NO_ID"
  logMsg("No SimBrief ID provided, normal use without SimBrief")
  return
end

sb_fetch_status = "LOADING"
logMsg("SimBrief recovery...")

local response_body = {}
local url = "https://www.simbrief.com/api/xml.fetcher.php?userid=" .. simbrief_id .. "&v=xml"

local res, code = http.request{
  url = url,
  sink = ltn12.sink.table(response_body),
  redirect = true
}

if code ~= 200 or not response_body or #response_body == 0 then
  sb_fetch_status = "ERROR"
  logMsg("SimBrief error (HTTP " .. tostring(code) .. "). The plugin continues without SimBrief.")
  return
end

local xml_body = table.concat(response_body)

sb_origin_icao    = string.match(xml_body, ".-(.-)") or ""
sb_dest_icao      = string.match(xml_body, ".-(.-)") or ""
sb_origin_name    = string.match(xml_body, ".-(.-)") or ""
sb_dest_name      = string.match(xml_body, ".-(.-)") or ""
sb_runway_takeoff = string.match(xml_body, ".-(.-)") or ""
sb_runway_landing = string.match(xml_body, ".-(.-)") or ""

if sb_origin_icao == "" or sb_dest_icao == "" then
  sb_fetch_status = "NO_DATA"
  logMsg("SimBrief does not contain valid data. The plugin continues normally.")
  return
end
```

```
sb_fetch_status = "OK"
logMsg("SimBrief OK - " .. sb_origin_icao .. " !' " .. sb_dest_icao)

sb_airport_mismatch = (sb_origin_icao ~= curr_ICAO)

if get_from_SimBrief then
  apply_simbrief_runway()
end
end

function apply_simbrief_runway()

if sb_fetch_status ~= "OK" then
  logMsg("No SimBrief runway application (status = " .. sb_fetch_status .. ")")
  return
end

local l_rwy = ""

if depart_arrive == 1 and sb_origin_icao == curr_ICAO then
  l_rwy = sb_runway_takeoff
elseif depart_arrive == 2 and sb_dest_icao == curr_ICAO then
  l_rwy = sb_runway_landing
end

if l_rwy == "" then
  logMsg("SimBrief did not provide a track for this mode.")
  return
end

for i = 1, #t_runway do
  if t_runway[i].ID == l_rwy then
    depart_runway = l_rwy
    logMsg("Runway SimBrief applied : " .. l_rwy)
    return
  end
end

logMsg("The SimBrief runway (" .. l_rwy .. ") does not exist at this airport. Ignored.")
end
```

```
function apply_simbrief_runway()

depart_runway = ""

if sb_fetch_status ~= "OK" then
  return
end

local l_rwy = ""

if depart_arrive == 1 and sb_origin_icao == curr_ICAO then

  l_rwy = sb_runway_takeoff
elseif depart_arrive == 2 and sb_dest_icao == curr_ICAO then

  l_rwy = sb_runway_landing
end

if l_rwy == "" then
  return
end

local l_found = false
for i = 1, #t_runway do
  if t_runway[i].ID == l_rwy then
    l_found = true
    break
  end
end

if l_found then
  depart_runway = l_rwy
else

  depart_runway = ""
  update_msg("-18")
end

end
```

```
function start_car()

car_speed = 0
car_accel = 0
tire_rotate = 0
steering = 0
car_sign = 0
remaining_dist_leg = 0
turning_is_active = 0
curr_node = 1

car_body_heading = t_node[1].heading
car_x = t_node[1].x
car_y = t_node[1].y
car_z = t_node[1].z

local l_car_is_in_front = false
local l_plane_is_in_front = false

l_car_is_in_front = chk_line_of_sight (fm_plane_head, 120, 120,  fm_plane_x, fm_plane_z, car_x, car_z)
l_plane_is_in_front = chk_line_of_sight (car_body_heading, 120, 120, car_x, car_z, fm_plane_x, fm_plane_z)

if l_car_is_in_front == false and depart_gate == 0 then
  update_msg( "6" )
else
  update_msg( "3" )
end

end

function determine_exit_angle( lever_angle)

local l_prev_deviation, l_curr_deviation = 8888, 0
local l_angle_btw_circle = 0
local l_circle_radius = 0
local l_dist_btw_circles = 0
local l_angle_horizon_to_center_circle2
local l_adjacent, l_hyper, l_x, l_z, l_delta_z = 0, 0, 0, 0, 0
local l_delta_angle_rad = 0
```

```
local l_delta_angle_deg = 0
local l_total_angle = 0

l_circle_radius = min_rot_radius
l_dist_btw_circles = 2 * l_circle_radius

for l_delta_angle_deg = lever_angle, lever_angle+90, 0.1 do

  l_delta_angle_rad = math.rad(l_delta_angle_deg)
  l_x = l_dist_btw_circles * math.sin(l_delta_angle_rad)
  l_adjacent = l_x + l_circle_radius
  l_z =  l_dist_btw_circles * math.cos(l_delta_angle_rad)
  l_delta_z = l_z - l_circle_radius
  l_hyper = math.sqrt( ( l_delta_z ^ 2) + ( l_adjacent ^ 2) )
  l_angle_horizon_to_center_circle2 = math.asin(l_delta_z / l_hyper)
  l_angle_horizon_to_center_circle2 = math.deg(l_angle_horizon_to_center_circle2)
  l_total_angle = math.asin(  l_circle_radius / l_hyper )
  l_total_angle = math.deg(l_total_angle)
  l_curr_deviation =  math.abs(l_total_angle - (l_angle_horizon_to_center_circle2 + lever_angle))

  if l_curr_deviation < l_prev_deviation then
    l_prev_deviation = l_curr_deviation
    l_angle_btw_circle = l_delta_angle_deg
  end
end

return 90 + l_angle_btw_circle

end

function chk_line_of_sight(shooter_heading, shooter_left_angle, shooter_right_angle, shooter_x, shooter_z,
target_x, target_z )

local l_heading_to_target, l_dist_to_target = 0, 0
local l_left_arc, l_right_arc = 0, 0
local l_within_sight = false

l_heading_to_target, l_dist_to_target = heading_n_dist( shooter_x, shooter_z, target_x, target_z)
l_left_arc = add_delta_clockwise (shooter_heading, shooter_left_angle , -1 )
l_right_arc = math.fmod ( shooter_heading + shooter_right_angle , 360)
```

```
if l_heading_to_target >= l_left_arc and l_heading_to_target <= l_left_arc + shooter_left_angle +
shooter_right_angle  then
  l_within_sight = true
elseif l_heading_to_target <= l_right_arc and l_heading_to_target >= l_right_arc - ( shooter_left_angle +
shooter_right_angle ) then
  l_within_sight = true
end

return l_within_sight, l_heading_to_target, l_dist_to_target

end

function move_car( in_dist, in_car_in_sight, in_car_is_behind)

if in_dist >= max_dist_from_plane and in_car_in_sight then
  if car_speed > 0 then
    if car_accel >= 0 then
      car_accel = deccel_avg
    end
  else
    car_accel = 0
  end

elseif in_dist >= avg_dist_from_plane and in_dist < max_dist_from_plane and in_car_in_sight then
  if car_speed > 0 then
    if fm_gnd_spd > car_speed then
      car_accel =   (fm_gnd_spd - car_speed) / elapsed_time
      if car_accel > accel_max then car_accel = accel_max end
    elseif car_accel >= 0 then
      car_accel = deccel_avg
    end
  else
    car_accel = 0
  end

elseif in_dist >= min_dist_from_plane and in_dist < avg_dist_from_plane and in_car_in_sight then
  if car_speed > 0 then
    if fm_gnd_spd  > car_speed then
      car_accel = accel_max
```

```
    elseif car_accel >= 0 then
      car_accel = accel_avg
    end
  else
    car_accel = accel_max
  end

elseif in_dist < min_dist_from_plane or in_car_is_behind == true then
  car_accel = accel_max

else
  if car_speed > 0 then
    car_accel = deccel_max
  else
    car_accel = 0
  end
end

manage_car_motion()

end

function manage_car_motion()

local l_dist = 0
local l_dist_stop = 0
local l_dist_turn = 0

car_speed = car_speed + ( car_accel * elapsed_time )

if car_speed > speed_max then
  car_speed = speed_max
  if car_accel > 0 then
    car_accel = 0
  end
elseif car_speed < 0 then
  car_speed = 0
end

if turning_is_active > 0 and t_node[curr_node + 1].speed ~= nil then
```

```
  if car_speed > t_node[curr_node + 1].speed then
    car_speed = t_node[curr_node + 1].speed
    if car_accel > 0 then
      car_accel = 0
    end
  end
end

l_dist =  ( car_speed * elapsed_time ) +  ( 0.5 *  math.abs( car_accel ) *  math.pow(elapsed_time, 2) )

if remaining_dist_leg > 0 then

  l_dist_stop =  math.abs(  0.5 * (  math.pow( (car_speed / 2 ), 2 ) / deccel_avg ) )

  l_dist_turn = 0
  if t_node[curr_node+1].dir ~= nil and t_node[curr_node+1].speed ~= nil and curr_node + 1 ~= #t_node then

    if turning_is_active == 0 and t_node[curr_node+1].dir ~= 0 and car_speed > t_node[curr_node+1].speed then
      l_dist_turn =  math.abs(  0.5 * ( ( ( ( car_speed + t_node[curr_node+1].speed ) / 2) ^ 2 ) / deccel_avg ) )

      if l_dist_turn > (remaining_dist_leg - l_dist ) then
        car_accel = deccel_avg
        car_speed = car_speed + ( car_accel * elapsed_time )
        l_dist =  ( car_speed * elapsed_time ) +  ( 0.5 *  math.abs( car_accel ) * math.pow(elapsed_time, 2) )
      end
    end
  end

  if curr_node + 1 == #t_node then
    if l_dist_stop > (remaining_dist_leg - l_dist )  then
      car_accel = deccel_avg
      car_speed = car_speed + ( car_accel * elapsed_time )
      l_dist =  ( car_speed * elapsed_time ) +  ( 0.5 *  math.abs( car_accel ) * math.pow(elapsed_time, 2) )
    end
  end

end

plot_position( l_dist )
```

```
end

function plot_position( in_act_dist )

local l_remaining_turn_dist = 0
local l_remaining_act_dist = 0
local l_remaining_leg_dist = 0
local l_remaining_rot = 0
local l_head1 = 0
local l_head2 = 0
local l_dist = 0
local l_heading_from_center, l_heading_to_center, l_radius = 0, 0, 0
local l_goto_nextnode = false
local l_AoR = 0
local l_act_dist = 0
local l_rear_wheel_heading = 0

if curr_node == #t_node then
  return
end

l_act_dist = in_act_dist

if turning_is_active == 0 then

  if curr_node + 1 ~= #t_node then
    remaining_dist_leg = 0
    l_rear_wheel_x, l_rear_wheel_z = coordinates_of_adjusted_ref(car_x, car_z, 0, car_rear_wheel_to_ref * -1,
car_body_heading )
    l_head1, remaining_dist_leg = heading_n_dist( l_rear_wheel_x, l_rear_wheel_z, t_node[curr_node+1].x,
t_node[curr_node+1].z)
  else
    l_head1 = t_node[curr_node].heading
  end

  if curr_node + 2 <= #t_node and t_node[curr_node+1].dir == nil then
    determine_dir_of_turn (l_head1, t_node[curr_node+1].heading, remaining_dist_leg)
  end

  t_node[curr_node].heading = l_head1
```

```
  steering = 0
  car_body_heading = l_head1

  if curr_node + 1 == #t_node then
    if in_act_dist > remaining_dist_leg then
      l_act_dist = remaining_dist_leg
    end
  elseif t_node[curr_node+1].dir ~= 0 then
    remaining_dist_leg = remaining_dist_leg  -  t_node[curr_node+1].dist_b4_turn
  end

  l_remaining_leg_dist = remaining_dist_leg - l_act_dist

  if l_remaining_leg_dist > 0 then
    car_x = car_x + math.sin ( math.rad ( l_head1) ) * l_act_dist
    car_z = car_z + math.cos ( math.rad ( l_head1) ) * l_act_dist * -1
    remaining_dist_leg = l_remaining_leg_dist
  end

  if l_remaining_leg_dist <= 0 then
    car_x = car_x + math.sin ( math.rad ( l_head1) ) * remaining_dist_leg
    car_z = car_z + math.cos ( math.rad ( l_head1) ) * remaining_dist_leg * -1
    remaining_dist_leg = 0
    l_act_dist = math.abs(l_remaining_leg_dist)

    if t_node[curr_node+1].dir ~= 0  and curr_node + 1 ~= #t_node then
      turning_is_active = 1
      t_node[curr_node+1].rot_x, t_node[curr_node+1].rot_z = CoR_coordinates_using_car_ref (car_x, car_z,
l_head1, t_node[curr_node+1].radius, t_node[curr_node+1].dir)
    else
      l_goto_nextnode = true
    end
  end

end

if turning_is_active == 1 then

  if t_node[curr_node+1].head1_exit == nil then
```

```
    l_head2, l_dist = heading_n_dist( t_node[curr_node+1].rot_x, t_node[curr_node+1].rot_z, t_node[curr_node+2].x,
t_node[curr_node+2].z)
    l_AoR = math.deg ( math.acos( t_node[curr_node+1].radius / l_dist ) )
    l_heading_from_center = minus_delta_clockwise (l_head2, l_AoR, t_node[curr_node+1].dir)
    t_node[curr_node+1].head1_exit = add_delta_clockwise(l_heading_from_center, 90, t_node[curr_node+1].dir)
    t_node[curr_node+1].heading = t_node[curr_node+1].head1_exit
  end

  l_remaining_rot = compute_angle_diff_dir ( car_body_heading , t_node[curr_node+1].head1_exit,
t_node[curr_node+1].dir )
  l_remaining_turn_dist = ( l_remaining_rot / 360 ) * ( 2 * math.pi *  t_node[curr_node+1].radius )
  l_remaining_act_dist = l_act_dist - l_remaining_turn_dist

  if l_remaining_act_dist <= 0 then
    l_AoR = 360 * l_act_dist / ( 2 * math.pi *  t_node[curr_node+1].radius )
  else
    l_AoR = l_remaining_rot
    l_act_dist = l_remaining_act_dist
    if t_node[curr_node+1].heading ~= t_node[curr_node+1].head1_exit then
      turning_is_active = 2
    else
      l_goto_nextnode = true
    end
  end

  car_body_heading = add_delta_clockwise(car_body_heading, l_AoR, t_node[curr_node+1].dir)
  l_heading_from_center = minus_delta_clockwise(car_body_heading, 90 - t_node[curr_node+1].angle_rear_to_ref,
t_node[curr_node+1].dir)
  car_x = t_node[curr_node+1].rot_x + math.sin ( math.rad ( l_heading_from_center ) ) *
t_node[curr_node+1].ref_rot_radius
  car_z = t_node[curr_node+1].rot_z + math.cos ( math.rad (  l_heading_from_center ) ) *
t_node[curr_node+1].ref_rot_radius * -1
  determine_steering (l_AoR, l_remaining_turn_dist, t_node[curr_node+1].dir, t_node[curr_node+1].steering)

  if turning_is_active == 2 then
    l_head1 = heading_n_dist( car_x, car_z, t_node[curr_node+2].x, t_node[curr_node+2].z)
    t_node[curr_node+1].rot2_x, t_node[curr_node+1].rot2_z = CoR_coordinates_using_car_ref (car_x, car_z,
car_body_heading, t_node[curr_node+1].radius, t_node[curr_node+1].dir * -1 )
    l_head2, l_dist = heading_n_dist( t_node[curr_node+1].rot2_x, t_node[curr_node+1].rot2_z,
t_node[curr_node+2].x, t_node[curr_node+2].z)
```

```
    l_AoR = math.deg ( math.acos( t_node[curr_node+1].radius / l_dist ) )
    l_heading_from_center = minus_delta_clockwise(l_head2, l_AoR, t_node[curr_node+1].dir * -1 )
    t_node[curr_node+1].head1_exit = add_delta_clockwise(l_heading_from_center, 90, t_node[curr_node+1].dir * -1
)
  end

end

if turning_is_active == 2 then

  l_remaining_rot = compute_angle_diff_dir ( car_body_heading , t_node[curr_node+1].head1_exit,
t_node[curr_node+1].dir * -1)
  l_remaining_turn_dist = ( l_remaining_rot / 360 ) * ( 2 * math.pi *  t_node[curr_node+1].radius )
  l_remaining_act_dist = l_act_dist - l_remaining_turn_dist

  if l_remaining_act_dist <= 0 then
    l_AoR = 360 * l_act_dist / ( 2 * math.pi *  t_node[curr_node+1].radius )
  else
    l_AoR = l_remaining_rot
    l_act_dist = l_remaining_act_dist
    l_goto_nextnode = true
  end

  car_body_heading = add_delta_clockwise(car_body_heading, l_AoR, t_node[curr_node+1].dir * -1)
  l_heading_from_center = minus_delta_clockwise(car_body_heading, 90 - t_node[curr_node+1].angle_rear_to_ref,
t_node[curr_node+1].dir * -1)
  car_x = t_node[curr_node+1].rot2_x + math.sin ( math.rad ( l_heading_from_center ) ) *
t_node[curr_node+1].ref_rot_radius
  car_z = t_node[curr_node+1].rot2_z + math.cos ( math.rad (  l_heading_from_center ) ) *
t_node[curr_node+1].ref_rot_radius * -1
  determine_steering (l_AoR, l_remaining_turn_dist, t_node[curr_node+1].dir * -1, t_node[curr_node+1].steering)

end

if l_goto_nextnode then

  turning_is_active = 0
  curr_node = curr_node + 1
  steering = 0
  car_sign = 0
```

```
  if curr_node < #t_node then
    remaining_dist_leg = 0
    l_head1, remaining_dist_leg = heading_n_dist( car_x, car_z, t_node[curr_node+1].x, t_node[curr_node+1].z)
    t_node[curr_node].heading = l_head1
    remaining_dist_leg = remaining_dist_leg - l_act_dist
  elseif curr_node == #t_node then
    remaining_dist_leg = 0
  end

end

car_y = probe_y (car_x, car_y, car_z)
tire_rotate = math.fmod( tire_rotate + ( in_act_dist * 360 / ( tire_diameter * math.pi ) ), 360)

if car_sign == 0 or curr_node >= #t_node - 2 then

  if curr_node ~= #t_node then
    if remaining_dist_leg < avg_dist_from_plane and t_node[curr_node+1].dir ~= nil and t_node[curr_node+1].AoC <
160 then
      if  t_node[curr_node+1].dir == -1 then
        car_sign = 3
      elseif  t_node[curr_node+1].dir == 1 then
        car_sign = 2
      end
    end

    if depart_arrive == 1 and t_node[curr_node].hotzone == "1" and curr_node >= #t_node - 2 then
      car_sign = 1
      if not string.find( Err_Msg[1].text, "Arrived at destination" ) then
        update_msg("5")
      end
    end

  else
    if depart_arrive ~= 1 then
      local l_gate_in_sight, l_to_gate_heading, _ = chk_line_of_sight ( car_body_heading, 45, 45,
      car_x, car_z,
      t_gate[arrival_gate].x, t_gate[arrival_gate].z )
      if l_gate_in_sight then
```

```
        car_sign = 1
      else
        local l_turn_dir = 0
        _, l_turn_dir = compute_angle_diff (car_body_heading, l_to_gate_heading)
        if l_turn_dir == 1 then
          car_sign = 2
        else
          car_sign = 3
        end
      end

      flightstart = 0
      update_msg("5")

    else
      car_sign = 1
    end
  end
end

end

function CoR_coordinates_using_car_ref (in_x, in_z, in_heading, in_radius, in_dir)
local l_angle_rear_to_ref = math.deg ( math.atan ( car_rear_wheel_to_ref / in_radius ) )
local l_ref_to_center_rot = math.sqrt ( (car_rear_wheel_to_ref ^ 2) + (in_radius ^ 2) )
local l_heading_to_center = add_delta_clockwise(in_heading, 90 + l_angle_rear_to_ref, in_dir)
local l_rot_x = in_x + math.sin ( math.rad ( l_heading_to_center ) ) * l_ref_to_center_rot
local l_rot_z = in_z + math.cos ( math.rad ( l_heading_to_center ) ) * l_ref_to_center_rot * -1
return l_rot_x, l_rot_z
end

function determine_dir_of_turn(in_head1, in_head2, in_dist)

local l_AoC = 0
local l_AoR = 0
local l_turn_radius = 0
local l_speed_skid = math.sqrt ( cof *  gravity * min_rot_radius )

l_AoR, t_node[curr_node+1].dir = compute_angle_diff (in_head1, in_head2 )
```

```
if t_node[curr_node+1].dir ~= 0 then

  l_AoC = 180 - l_AoR

  if l_AoC < 90 then
    t_node[curr_node+1].radius = min_rot_radius
    t_node[curr_node+1].speed = math.sqrt ( cof *  gravity * t_node[curr_node+1].radius )
    t_node[curr_node+1].dist_b4_turn =  t_node[curr_node+1].radius
    l_AoR = determine_exit_angle( 90 - l_AoC )
    t_node[curr_node+1].head1_exit = add_delta_clockwise(in_head1, l_AoR, t_node[curr_node+1].dir)
  else
    local l_speed_reduction_strength = 0
    local l_turn_speed = l_speed_skid + ( l_AoC - 90 ) / ( ( 180 - 90 ) / ( speed_max - l_speed_skid ) )

    if l_AoC < 140 then
      l_speed_reduction_strength =  math.exp ( ( l_AoC - 90 ) /  (10 + 6 * ( l_AoC % 90 ) / 10))
    else
      l_speed_reduction_strength =  math.exp ( ( 180 - l_AoC ) / 30 )
    end

    t_node[curr_node+1].speed =  l_turn_speed / l_speed_reduction_strength
    t_node[curr_node+1].radius = ( t_node[curr_node+1].speed ^ 2) / ( cof *  gravity )
    t_node[curr_node+1].dist_b4_turn =  t_node[curr_node+1].radius * math.tan ( math.rad ( l_AoR / 2) )

    local l_revised = false

    if t_node[curr_node+1].dist_b4_turn + 15 > in_dist then
      t_node[curr_node+1].dist_b4_turn = in_dist - 15
      if t_node[curr_node+1].dist_b4_turn < min_rot_radius + car_rear_wheel_to_ref then
        t_node[curr_node+1].dist_b4_turn = min_rot_radius + car_rear_wheel_to_ref
      end
      l_revised = true
    end

    if t_node[curr_node+1].dist_b4_turn + ( min_rot_radius + car_rear_wheel_to_ref ) > t_node[curr_node+1].dist
then
      t_node[curr_node+1].dist_b4_turn = t_node[curr_node+1].dist - ( min_rot_radius + car_rear_wheel_to_ref )
      if t_node[curr_node+1].dist_b4_turn < min_rot_radius * 2 + car_rear_wheel_to_ref  then
        t_node[curr_node+1].dist_b4_turn = min_rot_radius + car_rear_wheel_to_ref
      end
```

```
      l_revised = true
    end

    if l_revised == true then
      t_node[curr_node+1].radius = t_node[curr_node+1].dist_b4_turn / math.tan ( math.rad (  l_AoR / 2) )
      if t_node[curr_node+1].radius < min_rot_radius then
        t_node[curr_node+1].radius = min_rot_radius
      end
      t_node[curr_node+1].speed = math.sqrt ( cof *  gravity * t_node[curr_node+1].radius )
      if t_node[curr_node+1].speed > speed_max then
        t_node[curr_node+1].speed = speed_max
      end
    end
  end

  t_node[curr_node+1].angle_rear_to_ref = math.deg ( math.atan ( car_rear_wheel_to_ref /
t_node[curr_node+1].radius ) )
  t_node[curr_node+1].ref_rot_radius = math.sqrt ( (car_rear_wheel_to_ref ^ 2) + ( t_node[curr_node+1].radius ^ 2) )
  t_node[curr_node+1].steering = math.deg ( math.atan ( car_front_to_back_wheel / ( t_node[curr_node+1].radius -
width_btw_midtire / 2 ) ) )
  t_node[curr_node+1].AoC = l_AoC

end

end

function determine_steering (in_AoR, in_remaining_turn_dist, in_dir, in_steer_limit)

local l_prev_steer_angle = math.abs(steering)
local l_time_in_turn = ( in_AoR / 360 ) * ( 2 * math.pi *  t_node[curr_node+1].radius )
local l_steering_delta = l_time_in_turn * max_steering / 0.5

if l_steering_delta > in_steer_limit then
  l_steering_delta = in_steer_limit
end

local time_to_exit_turn = in_remaining_turn_dist / car_speed
local time_to_steer_neutral = l_prev_steer_angle * 0.5 / max_steering

if time_to_exit_turn < time_to_steer_neutral  then
```

```
  if in_dir  == 1 then
    steering = steering - l_steering_delta
    if steering <= 0 then steering = 0 end
  else
    steering = steering + l_steering_delta
    if steering >= 0 then steering = 0 end
  end
elseif l_prev_steer_angle < in_steer_limit and ( time_to_exit_turn > time_to_steer_neutral ) then
  if in_dir  == 1 then
    steering = steering + l_steering_delta
    if steering >= in_steer_limit then steering = in_steer_limit end
  else
    steering = steering - in_AoR
    if steering <= in_steer_limit * -1 then steering = in_steer_limit * -1 end
  end
end

end

function coordinates_of_adjusted_ref(in_ref_x, in_ref_z, in_delta_x, in_delta_z, in_heading )
local l_dist = math.sqrt ( (in_delta_x ^ 2) + (in_delta_z ^ 2) )
local l_heading = math.fmod ( ( math.deg ( math.atan2 ( in_delta_x , in_delta_z  ) ) + 360 ),  360 )
local l_shifted_x  =  in_ref_x - math.sin ( math.rad ( in_heading - l_heading) ) * l_dist * -1
local l_shifted_z  =  in_ref_z -  math.cos ( math.rad ( in_heading - l_heading) ) *  l_dist
return l_shifted_x, l_shifted_z
end

function heading_n_dist( in_from_x1, in_from_z1, in_to_x2, in_to_z2)
local l_heading = math.fmod ( ( math.deg ( math.atan2 ( in_to_x2 - in_from_x1 , - (in_to_z2 - in_from_z1)  ) ) + 360
),  360 )
local l_dist = math.sqrt( (( in_to_x2 - in_from_x1) ^ 2) +  (( in_to_z2 - in_from_z1) ^ 2) )
return l_heading, l_dist
end

function minus_delta_clockwise(in_heading, in_delta, in_direction)
local l_heading
if in_direction == 1 then
  l_heading = in_heading - in_delta
  if l_heading < 0 then l_heading = l_heading + 360 end
  return l_heading
```

```
elseif in_direction == -1 then
  return math.fmod (in_heading + in_delta, 360)
else
  return in_heading
end
end

function add_delta_clockwise(in_heading, in_delta, in_direction)
local l_heading = 0
if in_direction == 1 then
  return math.fmod (in_heading + in_delta, 360)
elseif in_direction == -1 then
  l_heading = in_heading - in_delta
  if l_heading < 0 then l_heading = l_heading + 360 end
  return l_heading
else
  return in_heading
end
end

function compute_angle_diff( in_from , in_to )
if in_to == in_from then
  return 0, 0
elseif in_to > in_from then
  if in_from + 180  > in_to  then
    return ( in_to - in_from ), 1
  else
    return ( in_from + (360 - in_to ) ), -1
  end
elseif in_to < in_from then
  if in_to + 180 > in_from then
    return ( in_from - in_to ), -1
  else
    return ( in_to + (360 - in_from ) ), 1
  end
end
end

function compute_angle_diff_dir ( in_from , in_to, in_dir )
if in_to == in_from then
```

```
  return 0
elseif in_dir > 0 then
  if in_to > in_from then
    return ( in_to - in_from )
  else
    return ( in_to + (360 - in_from ) )
  end
elseif in_dir < 0 then
  if in_to > in_from then
    return ( in_from + (360 - in_to ) )
  else
    return ( in_from - in_to )
  end
end
end

function local_to_latlon(l_x, l_y, l_z)
x1_value[0] = l_x
y1_value[0] = l_y
z1_value[0] = l_z
XPLM.XPLMLocalToWorld(x1_value[0],y1_value[0], z1_value[0], x1_value, y1_value, z1_value )
return  x1_value[0], y1_value[0], z1_value[0]
end

function latlon_to_local(in_lat, in_lon, in_alt)
x1_value[0] = in_lat
y1_value[0] = in_lon
z1_value[0] = in_alt
XPLM.XPLMWorldToLocal(x1_value[0],y1_value[0], z1_value[0], x1_value, y1_value, z1_value )
return  x1_value[0], y1_value[0], z1_value[0]
end

function get_local_coordinates(in_lat, in_lon, in_alt)
local l_x, l_y, l_z = 0, 0, 0
if l_alt == 0 then
  l_x, l_y, l_z = latlon_to_local(in_lat, in_lon, in_alt)
  x1_value[0] = l_x
  y1_value[0] = l_y
  z1_value[0] = l_z
  XPLM.XPLMProbeTerrainXYZ(proberef, x1_value[0], y1_value[0], z1_value[0], probeinfo_addr)
```

```
  probeinfo_value = probeinfo_addr
  in_lat, in_lon, in_alt = local_to_latlon(probeinfo_value[0].locationX, probeinfo_value[0].locationY,
probeinfo_value[0].locationZ)
end

l_x, l_y, l_z = latlon_to_local(in_lat, in_lon, in_alt)
x1_value[0] = l_x
y1_value[0] = l_y
z1_value[0] = l_z
XPLM.XPLMProbeTerrainXYZ(proberef, x1_value[0], y1_value[0], z1_value[0], probeinfo_addr)
probeinfo_value = probeinfo_addr
in_lat, in_lon, in_alt = local_to_latlon(probeinfo_value[0].locationX, probeinfo_value[0].locationY,
probeinfo_value[0].locationZ)
l_x, l_y, l_z = latlon_to_local(in_lat, in_lon, in_alt)
return l_x, l_y, l_z, in_alt
end

function probe_y (in_x, in_y, in_z)
local l_lat, l_on, l_alt = 0, 0, 0
x1_value[0] = in_x
y1_value[0] = in_y
z1_value[0] = in_z
XPLM.XPLMProbeTerrainXYZ(proberef, x1_value[0], y1_value[0], z1_value[0], probeinfo_addr)
probeinfo_value = probeinfo_addr
l_lat, l_lon, l_alt = local_to_latlon(probeinfo_value[0].locationX, probeinfo_value[0].locationY,
probeinfo_value[0].locationZ)
in_x, in_y, in_z = latlon_to_local(l_lat, l_lon, l_alt)
return in_y
end

function draw_object( in_x, in_y, in_z, in_heading)
dataref_float_addr = dataref_float_value
objpos_value[0].x = in_x
objpos_value[0].z = in_z
objpos_value[0].y = in_y

if car_accel == accel_max then
  objpos_value[0].pitch = 0.3
elseif car_accel == deccel_max then
  objpos_value[0].pitch = -0.3
```

```
else
  objpos_value[0].pitch = 0
end

objpos_value[0].heading = in_heading
objpos_value[0].roll = 0
objpos_value[0].structSize = ffi.sizeof(objpos_value[0])
objpos_addr = objpos_value

if obj_instance[0] ~= nil then
  XPLM.XPLMInstanceSetPosition(obj_instance[0], objpos_addr, dataref_float_addr)
end

dataref_float_value2[0] = car_sign
dataref_float_value2[1] = 0
dataref_float_addr = dataref_float_value2
objpos_value[0].y = in_y + place_above_the_car
objpos_value[0].heading = in_heading
objpos_value[0].x, objpos_value[0].z = coordinates_of_adjusted_ref(in_x, in_z, 0, place_Z_of_car, in_heading )
objpos_value[0].structSize = ffi.sizeof(objpos_value[0])
objpos_addr = objpos_value

if signboard_instance[0] ~= nil then
  XPLM.XPLMInstanceSetPosition(signboard_instance[0], objpos_addr, dataref_float_addr)
end
end

function object_physics()

local l_now = fm_run_time

if elapsed_time == 0 then
  elapsed_time = l_now
end

local l_dt = l_now - elapsed_time

if l_dt <= 0 or l_dt > 1 then
  elapsed_time = l_now
  return
```

```
end

elapsed_time = l_dt

if path_instance[0] ~= nil and path_is_shown == false then
  draw_path()
end

if rampstart_chg == true then
  if show_rampstart then
    load_rampstart()
  else
    unload_rampstart()
  end
  if rampstart_instance[0] ~= nil then
    draw_rampstart()
  end
end

if obj_instance[0] ~= nil and #t_node > 0 and curr_node ~= #t_node then
  local l_dist = 0
  local l_car_in_sight, l_car_is_behind = false, false
  l_car_in_sight, _, l_dist = chk_line_of_sight (fm_plane_head, 80, 80,  fm_plane_x, fm_plane_z, car_x, car_z)
  local l_car_to_plane_head, l_car_to_plane_dist = 0, 0
  if not l_car_in_sight and l_dist <= 200 then
    l_car_is_behind, l_car_to_plane_head, l_car_to_plane_dist = chk_line_of_sight (car_body_heading, 45, 45,
car_x, car_z, fm_plane_x, fm_plane_z)
  end
  move_car (l_dist, l_car_in_sight, l_car_is_behind)
  draw_object (car_x, car_y, car_z, car_body_heading)
end

elapsed_time = fm_run_time

end

function load_probe()
probeinfo_value[0].structSize = ffi.sizeof(probeinfo_value[0])
probeinfo_addr = probeinfo_value
probetype[1] = 0
```

```
proberef =  XPLM.XPLMCreateProbe(probetype[1])
end

function load_object()

ffi.copy(dataref_name,"sim/graphics/animation/ground_traffic/tire_steer_deg[0]")
dataref_array[0] = dataref_name
ffi.copy(dataref_name,"sim/graphics/animation/ground_traffic/tire_steer_deg[1]")
dataref_array[1] = dataref_name
ffi.copy(dataref_name,"sim/graphics/animation/ground_traffic/tire_rotation_angle_deg[0]")
dataref_array[2] = dataref_name
ffi.copy(dataref_name,"sim/graphics/animation/ground_traffic/tire_rotation_angle_deg[1]")
dataref_array[3] = dataref_name
ffi.copy(dataref_name,"sim/graphics/animation/ground_traffic/tire_rotation_angle_deg[2]")
dataref_array[4] = dataref_name
ffi.copy(dataref_name,"sim/graphics/animation/ground_traffic/tire_rotation_angle_deg[3]")
dataref_array[5] = dataref_name
dataref_array[6] = NULL
datarefs_addr = dataref_array

local l_auto_sel = 0
if car_type_fmcar == "Auto" then
  math.randomseed(os.time())
  l_auto_sel = math.random(1, 3)
end

if car_type_fmcar == "Ferrari" or (car_type_fmcar == "Auto" and l_auto_sel == 1) then
  XPLM.XPLMLoadObjectAsync( syspath .. "Resources/default scenery/airport
scenery/Dynamic_Vehicles/crew_car_ferrari.obj",
  function(inObject, inRefcon)
    obj_instance[0] = XPLM.XPLMCreateInstance(inObject, datarefs_addr)
    objref = inObject
    end,
    inRefcon )
    car_weight = 1445; tire_diameter = 0.79; min_turn_radius = 10.8
    width_btw_midtire = 1.86; car_front_to_back_wheel = 3.2; car_rear_wheel_to_ref = 1.60
    time_to_100 = 3; speed_max = 88.88; place_above_the_car = 1.43; place_Z_of_car = 0

  elseif car_type_fmcar == "Van" or (car_type_fmcar == "Auto" and l_auto_sel == 2) then
    XPLM.XPLMLoadObjectAsync( SCRIPT_DIRECTORY .. "follow_me/objects/fm_van.obj",
```

```
function(inObject, inRefcon)
  obj_instance[0] = XPLM.XPLMCreateInstance(inObject, datarefs_addr)
  objref = inObject
  end,
  inRefcon )
  car_weight = 1700; tire_diameter = 0.65; min_turn_radius = 10.8
  width_btw_midtire = 1.39; car_front_to_back_wheel = 2.64; car_rear_wheel_to_ref = 1.32
  time_to_100 = 10; speed_max = 70; place_above_the_car = 1.95; place_Z_of_car = -1.625

elseif car_type_fmcar == "Truck" or (car_type_fmcar == "Auto" and l_auto_sel == 3) then
  XPLM.XPLMLoadObjectAsync( SCRIPT_DIRECTORY .. "follow_me/objects/fm_truck.obj",
  function(inObject, inRefcon)
    obj_instance[0] = XPLM.XPLMCreateInstance(inObject, datarefs_addr)
    objref = inObject
    end,
    inRefcon )
    car_weight = 2000; tire_diameter = 0.730; min_turn_radius = 10.8
    width_btw_midtire = 1.578; car_front_to_back_wheel = 3.137; car_rear_wheel_to_ref = 1.5685
    time_to_100 = 7.5; speed_max = 54; place_above_the_car = 1.86; place_Z_of_car = -0.4347
  end

  ffi.copy(dataref_name,"fm/anim/sign")
  dataref_array2[0] = dataref_name
  dataref_array2[1] = NULL
  datarefs_addr = dataref_array2

  XPLM.XPLMLoadObjectAsync( SCRIPT_DIRECTORY .. "follow_me/objects/signboard.obj",
  function(inObject, inRefcon)
    signboard_instance[0] = XPLM.XPLMCreateInstance(inObject, datarefs_addr)
    signboardref = inObject
    end,
    inRefcon )
  end

  function load_path()
    if FM_car_active and show_path and #t_node > 0 then
      XPLM.XPLMLoadObjectAsync( SCRIPT_DIRECTORY .. "follow_me/objects/pushpin_yellow.obj",
      function(inObject, inRefcon)
        for i= 0, #t_node-1 do
          path_instance[i] = XPLM.XPLMCreateInstance(inObject, NULL)
```

```
    end
    pathref = inObject
    end,
    inRefcon )
  end
end

function load_rampstart()
  if rampstart_instance[0] == nil then
    XPLM.XPLMLoadObjectAsync( SCRIPT_DIRECTORY .. "follow_me/objects/diamond_marker.obj",
    function(inObject, inRefcon)
      rampstart_instance[0] = XPLM.XPLMCreateInstance(inObject, NULL)
      rampstartref = inObject
      end,
      inRefcon )
    end
  end

  function draw_path()
    local l_index = 0
    float_value[0] = 0
    float_addr = float_value
    for l_index = 0, #t_node-1 do
      objpos_value[0].x = t_node[l_index+1].x
      objpos_value[0].y = fm_plane_y
      objpos_value[0].z = t_node[l_index+1].z
      objpos_addr = objpos_value
      XPLM.XPLMInstanceSetPosition(path_instance[l_index], objpos_addr, float_addr)
    end
    path_is_shown = true
  end

  function draw_rampstart()
    local l_index = 0
    float_value[0] = 0
    float_addr = float_value

    if ( depart_arrive == 1 and depart_gate > 0 ) then
      l_index = depart_gate
    elseif  ( depart_arrive == 2 and arrival_gate > 0 ) then
```

```
    l_index = arrival_gate
  else
    l_index = 0
  end

  if l_index > 0 then
    objpos_value[0].x = t_gate[l_index].x
    objpos_value[0].y = t_gate[l_index].y
    objpos_value[0].z = t_gate[l_index].z
    objpos_value[0].heading = t_gate[l_index].Heading
  else
    objpos_value[0].x = 0.0
    objpos_value[0].y = fm_plane_y
    objpos_value[0].z =  0.0
  end

  objpos_addr = objpos_value
  XPLM.XPLMInstanceSetPosition(rampstart_instance[0], objpos_addr, float_addr)
  rampstart_chg = false
end

function unload_probe()
  if proberef ~= nil then XPLM.XPLMDestroyProbe(proberef) end
  proberef = nil
end

function unload_object()
  if obj_instance[0] ~= nil then XPLM.XPLMDestroyInstance(obj_instance[0]) end
  if objref ~= nil then XPLM.XPLMUnloadObject(objref) end
  obj_instance[0] = nil; objref = nil
  if signboard_instance[0] ~= nil then XPLM.XPLMDestroyInstance(signboard_instance[0]) end
  if signboardref ~= nil then XPLM.XPLMUnloadObject(signboardref) end
  signboard_instance[0] = nil; signboardref = nil
end

function unload_path()
  local l_index = 0
  if path_instance[0] ~= nil then
    for l_index = 0, #t_node-1 do
      XPLM.XPLMDestroyInstance(path_instance[l_index])
```

```
    end
  end
  if pathref ~= nil then XPLM.XPLMUnloadObject(pathref) end
  path_instance[0] = nil; pathref = nil; path_is_shown = false
end

function unload_rampstart()
  if rampstart_instance[0] ~= nil then XPLM.XPLMDestroyInstance(rampstart_instance[0]) end
  if rampstartref ~= nil then XPLM.XPLMUnloadObject(rampstartref) end
  rampstart_instance[0] = nil; rampstartref = nil; rampstart_chg = false
end

function register_dataref()

  dataref_register1 =
XPLM.XPLMRegisterDataAccessor("sim/graphics/animation/ground_traffic/tire_steer_deg[0]",2, 0,
  NULL, NULL,
  function(inRefcon) return steering end,
    NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);

    dataref_register7 =
XPLM.XPLMRegisterDataAccessor("sim/graphics/animation/ground_traffic/tire_steer_deg[1]",2, 0,
    NULL, NULL,
    function(inRefcon) return steering end,
      NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);

      dataref_register2 =
XPLM.XPLMRegisterDataAccessor("sim/graphics/animation/ground_traffic/tire_rotation_angle_deg[0]",2, 0,
      NULL, NULL,
      function(inRefcon) return tire_rotate end,
        NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);

        dataref_register3 =
XPLM.XPLMRegisterDataAccessor("sim/graphics/animation/ground_traffic/tire_rotation_angle_deg[1]",2, 0,
        NULL, NULL,
        function(inRefcon) return tire_rotate end,
          NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);

          dataref_register4 =
XPLM.XPLMRegisterDataAccessor("sim/graphics/animation/ground_traffic/tire_rotation_angle_deg[2]",2, 0,
```

```
NULL, NULL,
function(inRefcon) return tire_rotate end,
  NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);

  dataref_register5 =
XPLM.XPLMRegisterDataAccessor("sim/graphics/animation/ground_traffic/tire_rotation_angle_deg[3]",2, 0,
  NULL, NULL,
  function(inRefcon) return tire_rotate end,
    NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);

    dataref_register6 = XPLM.XPLMRegisterDataAccessor("fm/anim/sign",1, 0,
    function(inRefcon) return car_sign end,
      NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL);
    end

    function get_airport_elements()

      local l_airport_index = XPLMFindNavAid( nil, nil, LATITUDE, LONGITUDE, nil, xplm_Nav_Airport)
      local l_new_ICAO, l_new_ICAO_name =  "", ""

      _, _, _, _, _, _, l_new_ICAO, l_new_ICAO_name = XPLMGetNavAidInfo( l_airport_index )

      if curr_ICAO ~= l_new_ICAO then
        world_alt = 0
        taxiway_network = read_apt_file(l_new_ICAO)
        curr_ICAO = l_new_ICAO
        curr_ICAO_name = l_new_ICAO_name

        if taxiway_network ~= "" then
          XPLMSpeakString("FM Service is not available at this airport")
          return
        end

        if sb_fetch_status == "OK" then
          sb_airport_mismatch = (sb_origin_icao ~= curr_ICAO)
        end
      end

      if get_from_SimBrief then
        apply_simbrief_runway()
```

```
  end

  if #t_deleted_runway > 0 then
    update_msg("-18")
  end

  depart_gate = check_gate()
  if depart_arrive == 0 then
    if flightstart == 9999 then
      if depart_gate == 0 then
        depart_arrive = 2
      else
        flightstart = 0
        depart_arrive = 1
      end
    else
      depart_arrive = 1
    end
  end

end

function read_apt_file(in_ICAO)

  local l_filename1, l_filename2 = "", ""
  local l_file1, l_file2
  local l_line1, l_line2, l_rest = "", "", ""
  local l_str1 =  ""
  local l_airport_found = false
  local l_processed_1204 = false
  local l_processed_1300 = false
  local l_new_lines = ""
  local l_start = 0
  local l_not_first_line = false
  local l_terminate_loop = false

  l_filename1 = syspath .. "Custom Scenery/scenery_packs.ini"
  l_file1 = io.open( l_filename1 , "r")
  if l_file1 == nil then return "-11" end
```

```
repeat
  l_line1 = l_file1:read("*l")
  if l_line1 then
    l_str1 = string.match( l_line1,  "SCENERY_PACK (.*)" )
    if l_str1 then
      if l_str1 == "*GLOBAL_AIRPORTS*" then
        l_str1 = "Global Scenery/Global Airports/"
      end
      if string.find (l_str1, "") then
        l_filename2 = l_str1 .. "Earth nav data/apt.dat"
      else
        l_filename2 = syspath .. l_str1 .. "Earth nav data/apt.dat"
      end

      l_file2 = io.open( l_filename2 , "r")
      if l_file2 then
        while true do
          l_new_lines, l_rest = l_file2:read(BUFSIZE, "*l")
          if not l_new_lines then break end
          if l_rest then l_new_lines = l_new_lines .. l_rest .. 'n' end

          if l_airport_found == false then
            l_start = 0
            while true do
              if l_start > 0 then
                l_start = l_start + 200
              else
                l_start = 1
              end
              l_start = string.find(l_new_lines, in_ICAO, l_start)
              if l_start == nil then break end
              if l_start - 15 < 1 then
                l_start = 1
              else
                l_start = l_start - 15
              end
              l_new_lines = string.sub( l_new_lines, l_start)
              for l_line2 in l_new_lines:gmatch("[^rn]+") do
                if string.find(l_line2, in_ICAO) then
                  if string.match(l_line2, "^%d+ %s*[^%s]+%s*[^%s]+%s*[^%s]+%s*([^%s]+)%s*.*") ==
```

```
in_ICAO then
                    l_airport_found = true
                    l_not_first_line = false
                    initialise_airport()
                  end
                  break
                end
              end
              if l_airport_found == true then break end
            end
          end

          if l_airport_found then
            for l_line2 in l_new_lines:gmatch("[^rn]+") do
              if l_not_first_line == false then
                l_not_first_line = true
              else
                if string.find( l_line2,  "^1%s") or l_line2 == "99" then
                  table.sort(t_gate,function(a,b) return a.ID 0 then
    if depart_arrive == 1 and t_node[curr_node].hotzone == "1" and curr_node >= #t_node - 2
    and flightstart ~= 9999 then
      play_sound(snd_safeflight_bye)
    elseif depart_arrive == 2 and curr_node == #t_node then
      play_sound(snd_welcome_bye)
    end
  end
  t_node = {}
  Err_Msg[1] = {}; Err_Msg[2] = {}; Err_Msg[3] = {}
  depart_arrive = 0
  window_first_access = true
end

function handle_plugin_window()

  if ( fm_gear1_gnd == 0 and fm_gear2_gnd == 0 ) and fm_new_flight > 1  then
    if flightstart == 0 then
      flightstart = fm_run_time + 180
    end
    if fm_run_time > flightstart and flightstart ~= 9999 then
      arrival_gate = 0
```

```
    flightstart = 9999
    ground_time = 0
    prepare_kill_objects = true
  end
else
  if holder_wnd and toggle_window == true then
    window_is_open = not window_is_open
    if window_is_open then
      show_window()
    else
      hide_window()
    end
    toggle_window = false
  end
end

local l_err = ""

if FM_car_active == true and #t_node > 0 and prev_taxi_light ~= fm_taxi_light and fm_taxi_light ==
0 then
  if ( depart_arrive == 1 and t_node[curr_node].hotzone == "1" and curr_node >= #t_node - 2 ) or
  ( depart_arrive == 2 and curr_node == #t_node ) then
    prepare_kill_objects = true
  end
end

if FM_car_active == false and get_from_SimBrief == true
and fm_gear1_gnd == 1 and holder_wnd then
  if ( prev_taxi_light ~= fm_taxi_light and fm_taxi_light == 1 ) or
  ( prev_beacon_light ~= fm_beacon_light and fm_beacon_light == 1 ) then
    get_airport_elements()
    if taxiway_network == "" then
      if depart_runway ~= "" then
        l_err = determine_XP_route()
        if l_err == "" or ( l_err ~= "" and tonumber(l_err) > 0 ) then
          prepare_show_objects = true
        else
          update_msg(l_err)
        end
      end
```

```
      else
        update_msg("-15")
      end
    end
  end
  prev_taxi_light = fm_taxi_light
  prev_beacon_light = fm_beacon_light

  if FM_car_active == false and flightstart == 9999 and
  ( fm_gear1_gnd == 1 and fm_gear2_gnd == 1 ) and ground_time == 0 and
  fm_taxi_light == 1 and fm_gnd_spd < 15 then

    get_airport_elements()
    if taxiway_network == "" then

      if random_gate == true or arrival_gate > 0 then

        if arrival_gate == 0 then
          l_err = auto_assign_gate()
          if l_err == "1" then
            play_text = "No suitable gate for this aircraft. Randomly assigned to gate " ..
t_gate[arrival_gate].ID
          elseif l_err == "2" then
            play_text = "Assigned to ".. t_gate[arrival_gate].ID
          end
          play_time = fm_run_time + 5
        end

        if window_is_open == false then
          l_err = determine_XP_route()
          if l_err == "" or ( l_err ~= "" and tonumber(l_err) > 0 ) then
            prepare_show_objects = true
          else
            update_msg(l_err)
          end
        end
      end
    else
      update_msg("-15")
    end
```

```
  ground_time = 1
end

if play_time ~= 0 then
  if fm_run_time > play_time then
    XPLMSpeakString( play_text)
    play_text = ""
    play_time = 0
  end
end

if prepare_show_objects == true then
  FM_car_active = true
  load_object()
  load_path()
  start_car()
  rampstart_chg = true
  prepare_show_objects = false
end

if prepare_kill_objects == true then
  FM_car_active = false
  unload_object()
  unload_path()

  if kill_is_manual then
    unload_rampstart()
    rampstart_chg = false
    kill_is_manual = false

    local l_saved_runway       = depart_runway
    local l_saved_depart       = depart_arrive
    local l_saved_arrival_gate = arrival_gate
    local l_saved_gatetext     = gatetext
    local l_saved_depart_gate  = depart_gate
    initialise_routes()
    depart_runway  = l_saved_runway
    depart_arrive  = l_saved_depart
    arrival_gate   = l_saved_arrival_gate
    gatetext       = l_saved_gatetext
```

```
      depart_gate    = l_saved_depart_gate
      rampstart_chg  = true
    else
      if flightstart == 9999 then
        unload_rampstart()
      end
      initialise_routes()
    end
    prepare_kill_objects = false
  end

  if not holder_wnd and fm_new_flight > 1 then
    screen_width = SCREEN_WIDTH
    show_holder()
  end

  if holder_wnd and screen_width ~= SCREEN_WIDTH then
    screen_width = SCREEN_WIDTH
    float_wnd_set_position(holder_wnd, screen_width - Holder_len,  Win_Y)
    if followme_wnd ~= nil then
      float_wnd_set_position(followme_wnd, screen_width - 430,  Win_Y)
    end
  end

  if holder_wnd then
    if MOUSE_X >= SCREEN_WIDTH - Holder_len and MOUSE_X <= SCREEN_WIDTH and
    MOUSE_Y >= SCREEN_HIGHT - ( Win_Y + Holder_len ) and MOUSE_Y <= SCREEN_HIGHT -
Win_Y then
      float_wnd_bring_to_front(holder_wnd)
    end
    if holder_drag == 3 and  followme_wnd ~= nil then
      float_wnd_set_position(followme_wnd, screen_width - 430,  Win_Y)
      holder_drag = 2
    end
  end

  if followme_wnd ~= nil then
    if MOUSE_X >= SCREEN_WIDTH - 400 - Holder_len and MOUSE_X <= SCREEN_WIDTH -
Holder_len and
    MOUSE_Y >= SCREEN_HIGHT - ( Win_Y + 400 ) and MOUSE_Y <= SCREEN_HIGHT - Win_Y
```

```
then
        float_wnd_bring_to_front(followme_wnd)
      end
    end

  end

  function show_holder()
    if init_load == 0 then
      load_config()
      init_load = 1
    end
    holder_wnd = float_wnd_create(Holder_len, Holder_len, 2, true)
    float_wnd_set_position(holder_wnd, screen_width - Holder_len,  Win_Y)
    float_wnd_set_imgui_builder(holder_wnd, "build_holder")
    float_wnd_set_onclose(holder_wnd, "closed_holder")
  end

  function hide_holder()
    if holder_wnd then
      float_wnd_destroy(holder_wnd)
      holder_wnd = nil
    end
  end

  function show_window()
    window_first_access = true
    followme_wnd = float_wnd_create(400, 500, 2, true)
    float_wnd_set_position(followme_wnd, screen_width - 400 - Holder_len,  Win_Y)
    float_wnd_set_imgui_builder(followme_wnd, "build_window")
    float_wnd_set_onclose(followme_wnd, "closed_window")
  end

  function hide_window()
    if followme_wnd ~= nil then
      float_wnd_destroy(followme_wnd)
      followme_wnd = nil
      Window_is_inactive = true
    end
  end
```

```
function build_window(wnd, x, y)

  local l_err = ""
  local l_is_selected = false
  local l_changed = false
  local l_newtext = ""
  local l_newval = 0

  local l_flags = bit.bor (imgui.constant.WindowFlags.NoTitleBar,
  imgui.constant.WindowFlags.NoResize,
  imgui.constant.WindowFlags.NoMove,
  imgui.constant.WindowFlags.HorizontalScrollbar,
  imgui.constant.WindowFlags.NoSavedSettings
  )

  if window_first_access == true then
    get_airport_elements()
    l_err = taxiway_network
    window_first_access = false
  end

  if FM_car_active == true or #t_runway == 0 then
    imgui.PushStyleVar(imgui.constant.StyleVar.Alpha, 0.7)
    imgui.PushStyleVar(imgui.constant.StyleVar.WindowBorderSize, 0)
    if FM_car_active == false then imgui.SetNextWindowFocus() end
    if FM_car_active == true then
      imgui.SetNextWindowPos ( 0, 75 )
      imgui.SetNextWindowSize( 400, 120 )
    else
      imgui.SetNextWindowPos ( 0, 75 )
      imgui.SetNextWindowSize( 400, 170 )
    end
    if imgui.Begin("##Disabled", nil,l_flags) then imgui.End() end
    imgui.PopStyleVar()
    imgui.PopStyleVar()
    if FM_car_active == false then imgui.SetNextWindowFocus() end
  end

  imgui.SetWindowFontScale(1.1)
```

```
imgui.SetCursorPosY(5)
imgui.SetCursorPosX(10)
imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF888888)
imgui.TextUnformatted("Departure Airport")
imgui.PopStyleColor()

imgui.SetCursorPosY(18)
imgui.SetCursorPosX(10)
imgui.SetWindowFontScale(1.2)

if sb_fetch_status == "OK" and sb_origin_icao ~= "" then
  imgui.TextUnformatted(sb_origin_icao .. "   " .. sb_origin_name)
else

  imgui.TextUnformatted(curr_ICAO .. "   " .. curr_ICAO_name)
end

imgui.SetWindowFontScale(1.1)
imgui.SetCursorPosY(35)
imgui.SetCursorPosX(10)
imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF888888)
imgui.TextUnformatted("Arrival Airport")
imgui.PopStyleColor()

imgui.SetCursorPosY(48)
imgui.SetCursorPosX(10)
imgui.SetWindowFontScale(1.2)

if sb_fetch_status == "OK" and sb_dest_icao ~= "" then
  imgui.TextUnformatted(sb_dest_icao .. "   " .. sb_dest_name)
else
  imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF666666)
  imgui.TextUnformatted("---")
  imgui.PopStyleColor()
end

imgui.SetWindowFontScale(1.0)

imgui.SetCursorPosY(65)
imgui.Separator()
```

```
local l_y = 75

imgui.SetCursorPosY(l_y)
imgui.SetCursorPosX(18)
if imgui.RadioButton(" Departure", depart_arrive == 1, false) then
  depart_arrive = 1
  flightstart = 0
  rampstart_chg = true
  if get_from_SimBrief then apply_simbrief_runway() end
end

if depart_gate > 0 then
  imgui.SameLine()
  imgui.SetCursorPosX(160)
  imgui.TextUnformatted(t_gate[depart_gate].ID)
end

imgui.SetCursorPosY(l_y+25)
imgui.SetCursorPosX(48)
imgui.TextUnformatted("To Runway : ")

imgui.SameLine()
imgui.SetCursorPosX(130)
imgui.PushItemWidth(55)

if get_from_SimBrief then
  imgui.InputText("##text_runway", depart_runway, 5, imgui.constant.InputTextFlags.ReadOnly)
else
  local l_depart_runway = depart_runway
  if imgui.BeginCombo("##select_runway", l_depart_runway) then
    for i = 1, #t_runway  do
      l_is_selected = ( l_depart_runway == t_runway[i].ID)
      if imgui.Selectable( t_runway[i].ID, l_is_selected ) then
        depart_runway = t_runway[i].ID
        depart_arrive = 1
        flightstart = 0
      end
      if l_is_selected then imgui.SetItemDefaultFocus() end
    end
```

```
    imgui.EndCombo()
  end
end
imgui.PopItemWidth()

imgui.SameLine()
imgui.SetCursorPosX(230)

l_changed, l_newval = imgui.Checkbox("##Get SimBrief data", get_from_SimBrief)
if l_changed then
  get_from_SimBrief = l_newval
  if get_from_SimBrief then

    if simbrief_id ~= "" then
      check_SimBrief()
    else
      update_msg("-20")
    end
  else
    depart_runway = ""
  end
end

imgui.SameLine()
imgui.TextUnformatted("Get SimBrief data")

imgui.SetCursorPosY(l_y+60)
imgui.SetCursorPosX(18)

if imgui.RadioButton(" Arrival", depart_arrive == 2, false) then
  depart_arrive = 2
  rampstart_chg = true
  if get_from_SimBrief then apply_simbrief_runway() end
end

if depart_arrive == 2 and random_gate == true and arrival_gate == 0 then
  auto_assign_gate()
end

imgui.SameLine()
```

```
imgui.SetCursorPosX(230)

l_changed, l_newval = imgui.Checkbox("##Auto Assign", random_gate)
if l_changed then
  random_gate = l_newval
  if random_gate == true and arrival_gate == 0 then
    auto_assign_gate()
  end
end

imgui.SameLine()
imgui.TextUnformatted("Auto Assign")

imgui.SetCursorPosY(l_y+85)
imgui.SetCursorPosX(48)
imgui.TextUnformatted("To Gate/Ramp : ")

imgui.SameLine()
imgui.SetCursorPosX(160)
imgui.PushItemWidth(150)

l_changed, l_newtext = imgui.InputText("##filter_gate", gatetext, 30)
if l_changed then
  gatetext = l_newtext
  text_was_chg = true
end
imgui.PopItemWidth()

if imgui.IsItemClicked(0) then
  combo_filter_list = true
end

if combo_filter_list == true then
  imgui.SetNextWindowFocus()
  imgui.SetNextWindowPos ( 160, l_y + 105 )
  imgui.SetNextWindowSize( 150, 80 )
  if imgui.Begin("##combo_filter", nil, l_flags) then
    local l_gate = ""
    if arrival_gate > 0 then l_gate = t_gate[arrival_gate].ID end
    t_suitable_gates = {}
```

```
    for i = 1, #t_gate  do
      if  string.match(t_gate[i].Types, Aircraft_Type) == Aircraft_Type or
      ( Aircraft_Type == "0" and string.match(t_gate[i].Types, "7" ) ) then
        t_suitable_gates[#t_suitable_gates+1] = i
      end
    end
    if #t_suitable_gates == 0 then
      l_err = "-17"
      for i = 1, #t_gate  do t_suitable_gates[#t_suitable_gates+1] = i end
    end
    local l_into_list = true
    for i = 1, #t_suitable_gates  do
      if text_was_chg then
        if string.match( t_gate[t_suitable_gates[i]].ID, gatetext) == gatetext then
          l_into_list = true
        else
          l_into_list = false
        end
      end
      if l_into_list == true then
        l_is_selected = ( l_gate == t_gate[t_suitable_gates[i]].ID )
        if imgui.Selectable( t_gate[t_suitable_gates[i]].ID, l_is_selected ) then
          depart_arrive = 2
          arrival_gate = t_suitable_gates[i]
          gatetext = t_gate[arrival_gate].ID
          rampstart_chg = true
          combo_filter_list = false
          text_was_chg = false
        end
        if l_is_selected then imgui.SetItemDefaultFocus() end
      end
    end
    imgui.End()
  end
end

if  l_err == "-17" then
  if  Err_Msg[1].text ~= nil and string.find( Err_Msg[1].text, "No suitable gate for this plane. Lift" )
then
    l_err = ""
```

```
  end
end

imgui.SameLine()
if imgui.Button("X##clear_filter", 15, 15) then
  arrival_gate = 0; gatetext = ""; combo_filter_list = false
  text_was_chg = false; rampstart_chg = true
end

imgui.SetCursorPosY(l_y + 125)
imgui.SetCursorPosX(230)
l_changed, l_newval = imgui.Checkbox("##Limit Car Speed", speed_limiter)
if l_changed then
  speed_limiter = l_newval
  if speed_limiter == true then
    speed_max = 20
  else
    speed_max = car_default_speed
  end
end
imgui.SameLine()
imgui.TextUnformatted("Limit speed nto 20kts")

imgui.SetCursorPosY(l_y + 125)
imgui.SetCursorPosX(48)

if FM_car_active == false then
  if imgui.Button("Request Follow Me Car", 170, 30) then
    combo_filter_list = false
    l_err = determine_XP_route()
    if l_err == "" or ( l_err ~= "" and tonumber(l_err) > 0 ) then
      prepare_show_objects = true
    end
  end
else
  if imgui.Button("Cancel Follow Me Car", 170, 30) then
    prepare_kill_objects = true
    kill_is_manual = true
  end
end
```

```
imgui.SetCursorPosY(240)
imgui.Separator()

imgui.SetCursorPosY(250)
imgui.SetCursorPosX(20)
imgui.TextUnformatted("Model")
imgui.SameLine()
imgui.SetCursorPosX(90)
imgui.TextUnformatted("Type")
imgui.SameLine()
imgui.SetCursorPosX(160)
imgui.TextUnformatted("* Helps find suitable gate/ramp")

imgui.SetCursorPosY(270)
imgui.SetCursorPosX(20)
imgui.TextUnformatted(PLANE_ICAO)
imgui.SameLine()
imgui.SetCursorPosX(85)
imgui.TextUnformatted("*" )
imgui.SameLine()

local l_ac_types = ""

if Aircraft_Type == "" then
  l_ac_types = ""
else
  l_ac_types = ac_types[tonumber(Aircraft_Type) + 1]
end

if FM_car_active == true and depart_arrive == 2 then
  imgui.InputText("##text_actype", l_ac_types, 250, imgui.constant.InputTextFlags.ReadOnly)
else
  imgui.PushItemWidth(250)
  if imgui.BeginCombo("##select_type", l_ac_types) then
    for i = 1, #ac_types  do
      l_is_selected = ( l_ac_types == ac_types[i] )
      if imgui.Selectable(ac_types[i], l_is_selected) then
        l_ac_types = ac_types[i]
        Aircraft_Type = tostring(i - 1)
```

```
        gatetext = ""; arrival_gate = 0; rampstart_chg = true
        Err_Msg[1] = {}; Err_Msg[2] = {}; Err_Msg[3] = {}
        depart_gate = check_gate()
      end
      if l_is_selected then imgui.SetItemDefaultFocus() end
    end
    imgui.EndCombo()
  end
  imgui.PopItemWidth()
  imgui.SameLine()
  if imgui.Button("X##clear_type", 20, 20) then
    Aircraft_Type = ""; gatetext = ""; arrival_gate = 0; rampstart_chg = true
    Err_Msg[1] = {}; Err_Msg[2] = {}; Err_Msg[3] = {}
    depart_gate = check_gate()
  end
end

imgui.SetCursorPosY(305)
imgui.SetCursorPosX(20)
l_changed, l_newval = imgui.Checkbox(" Show Path", show_path)
if l_changed then
  show_path = l_newval
  if show_path then load_path() else unload_path() end
end

imgui.SetCursorPosY(330)
imgui.SetCursorPosX(20)
l_changed, l_newval = imgui.Checkbox(" Show Ramp Start", show_rampstart)
if l_changed then
  show_rampstart = l_newval
  rampstart_chg = true
end

imgui.SetCursorPosY(310)
imgui.SetCursorPosX(300)
if imgui.Button("Save Pref.", 80, 35) then
  l_err = save_config()
end

imgui.SetCursorPosY(360)
```

```
imgui.SetCursorPosX(20)
imgui.TextUnformatted("Vol ")
imgui.SameLine()
imgui.SetCursorPosX(60)
imgui.PushItemWidth(150)
l_changed, l_newval = imgui.SliderFloat("##Vol", vol, 1, 10, "%.0f")
imgui.PopItemWidth()
if l_changed then
  vol = l_newval
  set_sound_vol()
end
imgui.SetCursorPosY(355)
imgui.SetCursorPosX(220)
if imgui.Button("Test Vol", 65, 30) then
  play_sound(snd_welcome)
end

imgui.SetCursorPosX(20); imgui.SetCursorPosY(388)
if imgui.RadioButton(" Ferrari", car_type_fmcar == "Ferrari", false) then car_type_fmcar = "Ferrari"
end
imgui.SetCursorPosX(110); imgui.SetCursorPosY(388)
if imgui.RadioButton(" FM Van", car_type_fmcar == "Van", false) then car_type_fmcar = "Van" end
imgui.SetCursorPosX(195); imgui.SetCursorPosY(388)
if imgui.RadioButton(" FM Truck", car_type_fmcar == "Truck", false) then car_type_fmcar = "Truck"
end
imgui.SetCursorPosX(290); imgui.SetCursorPosY(388)
if imgui.RadioButton(" Auto Select", car_type_fmcar == "Auto", false) then car_type_fmcar = "Auto"
end

imgui.SetCursorPosY(413)
imgui.Separator()

imgui.SetCursorPosY(420)
imgui.SetCursorPosX(10)
imgui.TextUnformatted("SimBrief ID")

imgui.SameLine()
imgui.SetCursorPosX(95)
imgui.PushItemWidth(90)
l_changed, l_newtext = imgui.InputText("##simbrief_id", simbrief_id, 11,
```

```
                imgui.constant.InputTextFlags.CharsDecimal)
                if l_changed then

                  if string.len(l_newtext) <= 10 then
                    simbrief_id = l_newtext
                  else
                    simbrief_id = string.sub(l_newtext, 1, 10)
                  end
                end
                imgui.PopItemWidth()

                imgui.SameLine()
                imgui.SetCursorPosX(195)
                if imgui.Button("Fetch SimBrief", 120, 20) then
                  if simbrief_id ~= "" then
                    check_SimBrief()
                  else
                    update_msg("-20")
                  end
                end

                imgui.SetCursorPosY(443)
                imgui.SetCursorPosX(10)
                if sb_fetch_status == "OK" then
                  if get_from_SimBrief and sb_airport_mismatch then
                    imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF0000FF)
                    imgui.TextUnformatted("No SimBrief data for the current airport")
                    imgui.PopStyleColor()
                  else
                    imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF00FF00)
                    imgui.TextUnformatted("SimBrief Runways  Dep:" .. sb_runway_takeoff .. "  Arr:" ..
sb_runway_landing)
                    imgui.PopStyleColor()
                  end
                elseif sb_fetch_status == "LOADING" then
                  imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF00FFFF)
                  imgui.TextUnformatted("Fetching SimBrief data...")
                  imgui.PopStyleColor()
                elseif sb_fetch_status == "ERROR" then
                  imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF0000FF)
```

```
    imgui.TextUnformatted("SimBrief fetch error")
    imgui.PopStyleColor()
  end

  imgui.SetCursorPosY(463)
  imgui.Separator()

  if l_err ~= "" then
    update_msg(l_err)
  end

  local l_y2 = 16
  for i= 1, 3 do
    imgui.SetCursorPosY(458 + l_y2 * i)
    imgui.SetCursorPosX(20)
    if not Err_Msg[i].text then break end
    if i == 1 then
      if Err_Msg[i].color == "RED" then
        imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF0000FF )
      else
        imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF00FF00 )
      end
    else
      imgui.PushStyleColor(imgui.constant.Col.Text, 0xFF696969 )
    end
    imgui.TextUnformatted( Err_Msg[i].text )
    imgui.PopStyleColor()
  end

end

function set_sound_vol ()
  set_sound_gain(snd_arrived, vol/10)
  set_sound_gain(snd_followme, vol/10)
  set_sound_gain(snd_safeflight_bye, vol/10)
  set_sound_gain(snd_welcome, vol/10)
  set_sound_gain(snd_welcome_bye, vol/10)
end

function update_msg(in_msg)
```

```
if in_msg == nil then return end

if tonumber(in_msg) ~= nil and tonumber(in_msg) < 0 then
  Err_Msg[1].color = "RED"
else
  Err_Msg[1].color = "GREEN"
end

if in_msg == "-18" then
  in_msg = "Routes not defined for runway "
  for i=1, #t_deleted_runway do
    if i == #t_deleted_runway then
      in_msg = in_msg.. t_deleted_runway[i]
    else
      in_msg = in_msg.. t_deleted_runway[i].. ", "
    end
  end
  if Err_Msg[1].text ~= nil and Err_Msg[1].text == in_msg then return end
end

if Err_Msg[1].text then
  Err_Msg[3].text = Err_Msg[2].text
  Err_Msg[2].text = Err_Msg[1].text
end

if in_msg == "6" then
  in_msg = "Follow Me Car ready. Car is behind you."
  if depart_arrive == 1 then play_sound(snd_followme) else play_sound(snd_welcome) end
elseif in_msg == "5" then
  in_msg = "Arrived at destination"
  play_sound(snd_arrived)
elseif in_msg == "4" then
  in_msg = "No route found. Remove taxiway limitation, trying again."
elseif in_msg == "3" then
  in_msg = "Follow Me Car is ready"
  if depart_arrive == 1 then play_sound(snd_followme) else play_sound(snd_welcome) end
elseif in_msg == "2" then
  in_msg = "Preference Saved"
elseif in_msg == "-1" then
```

```
    in_msg = "Specify the Aircraft Type"
  elseif in_msg == "-2" then
    in_msg = "Preference file not found. Apply default values."
  elseif in_msg == "-4" then
    in_msg = "Unable to complete Save operation"
  elseif in_msg == "-5" then
    if get_from_SimBrief then
      in_msg = "No runway from SimBrief data"
    else
      in_msg = "Select depart runway"
    end
  elseif in_msg == "-6" then
    in_msg = "Select a gate/ramp"
  elseif in_msg == "-11" then
    in_msg = "Unable to locate scenery_packs.ini"
  elseif in_msg == "-12" then
    in_msg = "Unable to find a suitable start pt"
  elseif in_msg == "-13" then
    in_msg = "Unable to find a suitable end pt"
  elseif in_msg == "-14" then
    in_msg = "Unable to find a suitable route"
  elseif in_msg == "-15" then
    in_msg = "Taxi Routes not defined for this airport"
  elseif in_msg == "-16" then
    in_msg = "Can't find start pt. Get off runway and request again"
  elseif in_msg == "-17" then
    in_msg = "No suitable gate for this plane. Lift restriction."

  elseif in_msg == "-20" then
    in_msg = "No SimBrief ID configured. Enter your ID below."
  elseif in_msg == "-21" then
    in_msg = "SimBrief connection failed. Check network."
  end

  Err_Msg[1].text = in_msg

end

function determine_XP_route()
```

```
    if #t_taxinode == 0 then return "-15" end
    if Aircraft_Type == "" then return "-1" end
    if depart_arrive == 0 then return "-5" end
    if depart_arrive == 2 and arrival_gate == 0 then return "-6" end

    if depart_arrive == 1 then
      if depart_runway == "" then return "-5" end

      local l_found = false
      for i = 1, #t_runway  do
        if t_runway[i].ID == depart_runway then
          l_found = true
          break
        end
      end
      if not l_found then return "-18" end
    end

    depart_gate = check_gate()
    return determine_possible_routes()

  end

  function determine_possible_routes()

    local l_startpt_node, l_startpt_x, l_startpt_z = 0, 0, 0, 0
    local l_endpt_node, l_endpt_x, l_endpt_z = 0, 0, 0
    local l_index = 0
    local l_found = false
    local l_rev_heading = add_delta_clockwise( fm_plane_head, 180, 1)

    if depart_gate ~= 0 then
      l_found, l_startpt_node, l_startpt_x, l_startpt_z = determine_pos_on_segment (
t_gate[depart_gate].Heading,
      t_gate[depart_gate].x,
      t_gate[depart_gate].z,
      t_gate[depart_gate].Ramptype )
    else
      local l_adj_fm_plane_x = 0
      local l_adj_fm_plane_z = 0
```

```
      l_adj_fm_plane_x, l_adj_fm_plane_z = coordinates_of_adjusted_ref( fm_plane_x, fm_plane_z, 0,
60, fm_plane_head )
      l_found, l_startpt_node, l_startpt_x, l_startpt_z = determine_pos_on_segment ( fm_plane_head,
      l_adj_fm_plane_x,
      l_adj_fm_plane_z,
      "tie_down" )
      if l_found == false then
        l_found, l_startpt_node, l_startpt_x, l_startpt_z = determine_pos_on_segment ( l_rev_heading,
fm_plane_x, fm_plane_z, "tie_down")
      end
      if l_found == false and depart_arrive == 2 then return "-16" end
    end

    if l_found == false then return "-12" end

    if depart_arrive == 1 then
      for l_index = 1, #t_runway do
        if t_runway[l_index].ID == depart_runway then
          l_endpt_node = t_runway[l_index].Node
          break
        end
      end
    else
      l_found, l_endpt_node, l_endpt_x, l_endpt_z = determine_pos_on_segment(
t_gate[arrival_gate].Heading,
      t_gate[arrival_gate].x,
      t_gate[arrival_gate].z,
      "gate")
      if l_found == false then return "-13" end
    end

    impose_restriction_chk = true

    for l_index = 1, #t_taxinode do
      _, t_taxinode[l_index].h_value = heading_n_dist( t_taxinode[l_index].x, t_taxinode[l_index].z,
      t_taxinode[l_endpt_node+1].x, t_taxinode[l_endpt_node+1].z )
    end

    if depart_gate ~= 0 then
      transverse(l_startpt_node, l_endpt_node, -1)
```

```
  else
    transverse(l_startpt_node, l_endpt_node, fm_plane_head)
    if #t_possible_route == 0 then
      transverse(l_startpt_node, l_endpt_node, -1)
    end
  end

  if #t_possible_route == 0 then
    update_msg("4")
    impose_restriction_chk = false
    transverse(l_startpt_node, l_endpt_node, -1)
    if #t_possible_route == 0 then return "-14" end
  end

  process_possible_routes()
  return ""

end

function transverse(in_startnode, in_endnode, in_heading)

  function evaluate_node(in_node, in_size, t_open, t_close)
    local l_in_open, l_in_close = false, false
    for i = 1, #t_open do
      if t_open[i].Node == in_node then l_in_open = true; break end
    end
    for i = 1, #t_close do
      if t_close[i].Node == in_node then l_in_close = true; break end
    end
    if l_in_open == true or l_in_close == true then return false end
    if not impose_restriction_chk then return true
  else
    if in_size ~= "" then
      local l_aircraft_type = tonumber(Aircraft_Type)
      if in_size == "A"  then
        if ( l_aircraft_type > 0 and l_aircraft_type < 7 ) then return false end
      elseif ( in_size == "B" or in_size == "C" or in_size == "D" ) then
        if ( l_aircraft_type > 0 and l_aircraft_type < 3 ) then return false end
      elseif in_size == "E"  then
        if ( l_aircraft_type == 1 ) then return false end
```

```
      end
    end
    return true
  end
  return false
end

local t_close, t_open = {},{}
local l_pass = false
local l_curr_node, l_node, l_idx, l_curr_heading, l_AoC = 0, 0, 0 ,0, 0
local l_segment_idx, l_g_value, l_f_value = 0, 0, 0
local l_string = ""

t_possible_route = {}
for l_idx = 1, #t_taxinode do
  t_taxinode[l_idx].f_value = nil
  t_taxinode[l_idx].g_value = nil
  t_taxinode[l_idx].parent = nil
  t_taxinode[l_idx].cost = nil
  t_taxinode[l_idx].heading = nil
end

t_taxinode[in_startnode+1].f_value = t_taxinode[in_startnode+1].h_value
if in_heading ~= -1 then t_taxinode[in_startnode+1].heading = in_heading end
l_curr_node = in_startnode

t_open[1] = {}
t_open[1].Node = in_startnode
t_open[1].f_value = t_taxinode[in_startnode+1].f_value

while l_curr_node ~= in_endnode and l_curr_node ~= -1 do
  for l_string in t_taxinode[l_curr_node+1].Segment:gmatch("[^,]+") do
    l_segment_idx = tonumber(l_string)
    l_pass = false
    if t_segment[l_segment_idx].Type ~= "runway" or ( t_segment[l_segment_idx].Type == "runway"
and not impose_restriction_chk ) then
      if t_segment[l_segment_idx].Node1 == l_curr_node then
        l_node = t_segment[l_segment_idx].Node2
        l_pass = evaluate_node( l_node, t_segment[l_segment_idx].Size, t_open, t_close )
        l_curr_heading = t_segment[l_segment_idx].Heading
```

```
                elseif ( t_segment[l_segment_idx].Node2 == l_curr_node and
t_segment[l_segment_idx].Dir=="twoway" ) then
                  l_node = t_segment[l_segment_idx].Node1
                  l_pass = evaluate_node( l_node, t_segment[l_segment_idx].Size, t_open, t_close )
                  l_curr_heading = add_delta_clockwise(t_segment[l_segment_idx].Heading, 180, 1)
                end
              end

              if l_pass then
                l_idx = #t_open+1
                t_open[l_idx] = {}
                t_open[l_idx].Node = l_node
                if t_taxinode[l_curr_node+1].g_value == nil then
                  l_g_value = t_segment[l_segment_idx].Dist
                else
                  l_g_value = t_taxinode[l_curr_node+1].g_value + t_segment[l_segment_idx].Dist
                end
                l_f_value = l_g_value + t_taxinode[l_node+1].h_value
                if t_taxinode[l_node+1].f_value == nil  or l_f_value < t_taxinode[l_node+1].f_value then
                  t_taxinode[l_node+1].g_value = l_g_value
                  t_taxinode[l_node+1].f_value = l_f_value
                  t_taxinode[l_node+1].parent = l_curr_node
                  if t_taxinode[l_curr_node+1].heading == nil then
                    t_taxinode[l_node+1].cost = 0
                  else
                    if t_taxinode[l_curr_node+1].cost == nil then t_taxinode[l_curr_node+1].cost = 0 end
                    l_AoC = 180 - compute_angle_diff (t_taxinode[l_curr_node+1].heading, l_curr_heading )
                    if l_AoC < 20 then t_taxinode[l_node+1].cost = t_taxinode[l_curr_node+1].cost + 150
                  elseif l_AoC >= 20 and l_AoC <=80 then t_taxinode[l_node+1].cost =
t_taxinode[l_curr_node+1].cost + 12
                  elseif l_AoC > 80 and l_AoC < 90 then t_taxinode[l_node+1].cost =
t_taxinode[l_curr_node+1].cost + 3
                  elseif l_AoC >= 90 and l_AoC <= 140 then t_taxinode[l_node+1].cost =
t_taxinode[l_curr_node+1].cost + 1
                  elseif l_AoC > 140 and l_AoC <= 170 then t_taxinode[l_node+1].cost =
t_taxinode[l_curr_node+1].cost + 1
                  elseif l_AoC > 170 then t_taxinode[l_node+1].cost = t_taxinode[l_curr_node+1].cost  end
                  end
                  t_taxinode[l_node+1].heading = l_curr_heading
                  t_open[l_idx].f_value = l_f_value
```

```
        t_open[l_idx].cost = t_taxinode[l_node+1].cost
      end
    end
  end

  for l_idx = #t_open, 1, -1 do
    if t_open[l_idx].Node == l_curr_node then
      table.remove(t_open, l_idx)
      break
    end
  end
  l_idx = #t_close+1
  t_close[l_idx] = {}
  t_close[l_idx].Node = l_curr_node

  table.sort( t_open, function(a,b) return a.f_value > b.f_value end)
  for l_idx = #t_open, 1, -1 do
    if l_idx == #t_open then
      l_f_value = t_open[l_idx].f_value
      t_open[l_idx].order = 1
    else
      if math.abs(t_open[l_idx].f_value - l_f_value) < 300 then
        t_open[l_idx].order = 1
      else
        t_open[l_idx].order = 2
      end
    end
  end
  table.sort( t_open, function(a,b) return a.order > b.order or (a.order == b.order and a.cost > b.cost)
end)
  if #t_open > 0 then
    l_curr_node = t_open[#t_open].Node
  else
    l_curr_node = -1
  end
end

if t_taxinode[in_endnode+1].f_value ~= nil then
  t_possible_route[1] = {}
  t_possible_route[1].Dist = t_taxinode[in_endnode+1].g_value
```

```
    t_possible_route[1].Cost = t_taxinode[in_endnode+1].cost
    l_node = in_endnode
    t_possible_route[1].Route = in_endnode
    while l_node ~= in_startnode do
      l_node = t_taxinode[l_node+1].parent
      t_possible_route[1].Route = l_node .." ".. t_possible_route[1].Route
    end
  end
end

function process_possible_routes()
  if #t_possible_route == 0 then return end

  local l_bypass = false
  local l_new_head, l_new_dist = 0, 0
  local l_new_AoR = 0
  local l_index = 1
  local l_node = 0
  local l_hotzone_node_cnt = 0
  local l_string = ""
  local l_segment_idx = 0
  t_node = {}
  local l_min_dist_btw_nodes = min_rot_radius * 2 + car_rear_wheel_to_ref
  local t_route_nodes = {}
  local routenode_cnt = 1
  local l_last_taxiway = 0

  for l_node in t_possible_route[1].Route:gmatch("[^%s]+") do
    t_route_nodes[routenode_cnt] = l_node
    routenode_cnt = routenode_cnt + 1
  end

  if depart_arrive == 1 then
    for routenode_cnt = #t_route_nodes - 1 , 1, -1 do
      l_node = t_route_nodes[routenode_cnt]
      if t_taxinode[l_node+1].Type == "hotzone" or t_taxinode[l_node+1].Type == "runway" then
        for l_string in t_taxinode[l_node+1].Segment:gmatch("[^,]+") do
          l_segment_idx = tonumber(l_string)
          if t_segment[l_segment_idx].Hotzone ~= "" or t_segment[l_segment_idx].Type == "runway" then
          else
```

```
                l_last_taxiway = routenode_cnt
                break
              end
            end
          end
          if l_last_taxiway > 0 then break end
        end
      end

      for routenode_cnt = 1, #t_route_nodes do
        l_node = t_route_nodes[routenode_cnt]
        l_bypass = false
        if depart_arrive == 1 and routenode_cnt >= l_last_taxiway then
          if t_taxinode[l_node+1].Type == "hotzone" or t_taxinode[l_node+1].Type == "runway" then
            l_hotzone_node_cnt = l_hotzone_node_cnt + 1
          end
        end
        if l_hotzone_node_cnt > 2 then break end
        if l_bypass == false  then
          if l_index > 1  then
            l_new_head, l_new_dist = heading_n_dist( t_node[l_index-1].x, t_node[l_index-1].z,
            t_taxinode[l_node + 1].x, t_taxinode[l_node + 1].z )
            if l_index > 2 and l_hotzone_node_cnt == 0 then
              l_new_AoR = compute_angle_diff( t_node[l_index-2].heading , l_new_head )
              if l_new_AoR <= 10 then
                t_node[l_index-1].x = t_taxinode[l_node + 1].x
                t_node[l_index-1].y = t_taxinode[l_node + 1].y
                t_node[l_index-1].z = t_taxinode[l_node + 1].z
                t_node[l_index-2].heading, t_node[l_index-2].dist = heading_n_dist( t_node[l_index-2].x,
t_node[l_index-2].z,
                t_node[l_index-1].x, t_node[l_index-1].z )
                l_bypass = true
              end
            end
          end
          if l_bypass == false then
            t_node[l_index] = {}
            if l_hotzone_node_cnt > 0 then t_node[l_index].hotzone = "1" else t_node[l_index].hotzone = ""
end
            t_node[l_index].x = t_taxinode[l_node + 1].x
```

```
                t_node[l_index].y = t_taxinode[l_node + 1].y
                t_node[l_index].z = t_taxinode[l_node + 1].z
                if l_index > 1 then
                  t_node[l_index-1].heading = l_new_head
                  t_node[l_index-1].dist = l_new_dist
                end
                l_index = l_index + 1
              end
            end
          end

          for l_index = 1, #t_node - 2 do
            if t_node[l_index].dist < l_min_dist_btw_nodes then
              t_node[l_index+1].x, t_node[l_index+1].z = coordinates_of_adjusted_ref(t_node[l_index+1].x,
t_node[l_index+1].z,
              0, ( l_min_dist_btw_nodes - t_node[l_index].dist ),
              t_node[l_index].heading )
              t_node[l_index].dist = l_min_dist_btw_nodes
              t_node[l_index+1].heading, t_node[l_index+1].dist = heading_n_dist( t_node[l_index+1].x,
t_node[l_index+1].z,
              t_node[l_index+2].x, t_node[l_index+2].z )
            end
          end

          while t_taxinode[#t_taxinode].Type == "New" do t_taxinode[#t_taxinode] = nil end

            while t_segment[#t_segment].ID == "ADD_NEWSEGMENT" do
              if t_taxinode[t_segment[#t_segment].Node1 + 1] ~= nil then
                t_taxinode[t_segment[#t_segment].Node1 + 1].Segment =
string.gsub(t_taxinode[t_segment[#t_segment].Node1 + 1].Segment, ","..tostring(#t_segment), "")
              end
              if t_taxinode[t_segment[#t_segment].Node2 + 1] ~= nil then
                t_taxinode[t_segment[#t_segment].Node2 + 1].Segment =
string.gsub(t_taxinode[t_segment[#t_segment].Node2 + 1].Segment, ","..tostring(#t_segment), "")
              end
              t_segment[#t_segment] = nil
            end
          end

          function determine_pos_on_segment ( in_heading, in_x, in_z, in_type )
```

```
local l_within_sight, l_intersect_x, l_intersect_z, l_dist_to_intersect = false, 0, 0, 0
local l_ret_intersect_dist = 9999
local l_ret_segment_index, l_ret_x, l_ret_z = 0, 0, 0
local l_deadnode, l_dist_to_deadnode
local l_ret_node_dist = 9999
local l_ret_nodesegment_index, l_ret_deadnode = 0, 0
local l_in_rev_heading = 0
local l_goto_node = 0
local l_bypass = false

local l_min_dist = 25
local l_max_dist = 300
local l_max_dist_node = 130
local l_aircraft_type = tonumber(Aircraft_Type)

if l_aircraft_type == 0 or l_aircraft_type >= 7 then
  l_min_dist = 5
elseif l_aircraft_type >= 3 and l_aircraft_type < 7 then
  l_min_dist = 15
elseif l_aircraft_type == 2 then
  l_min_dist = 30
elseif l_aircraft_type == 1 then
  l_min_dist = 30
end

for i = 1, #t_segment do
  if ( in_type == "gate" or in_type == "misc" ) then
    l_within_sight, l_intersect_x, l_intersect_z, l_dist_to_intersect = compute_intersection (in_type,
in_heading, in_x, in_z, i)
    if l_within_sight and l_dist_to_intersect <= l_max_dist then
      if l_dist_to_intersect <= l_ret_intersect_dist then
        l_ret_intersect_dist = l_dist_to_intersect
        l_ret_segment_index = i
        l_ret_x = l_intersect_x
        l_ret_z = l_intersect_z
      end
    end
    l_within_sight, l_deadnode, l_dist_to_deadnode = check_deadend_node (in_type, in_heading,
in_x, in_z, i)
```

```
            if l_within_sight then
              if l_dist_to_deadnode >= l_min_dist and l_dist_to_deadnode <= l_ret_node_dist then
                l_ret_deadnode = l_deadnode
                l_ret_nodesegment_index = i
                l_ret_node_dist = l_dist_to_deadnode
              end
            end
          end

          if ( in_type == "tie_down" or in_type == "hangar" ) then
            l_within_sight, l_deadnode, l_dist_to_deadnode = check_deadend_node (in_type, in_heading,
in_x, in_z, i)
            if l_within_sight then
              if l_dist_to_deadnode <= l_max_dist_node and l_dist_to_deadnode <= l_ret_node_dist then
                l_ret_deadnode = l_deadnode
                l_ret_nodesegment_index = i
                l_ret_node_dist = l_dist_to_deadnode
              end
            end
            l_within_sight, l_intersect_x, l_intersect_z, l_dist_to_intersect = compute_intersection (in_type,
in_heading, in_x, in_z, i)
            if l_within_sight and l_dist_to_intersect <= l_max_dist then
              if l_dist_to_intersect <= l_ret_intersect_dist then
                l_ret_intersect_dist = l_dist_to_intersect
                l_ret_segment_index = i
                l_ret_x = l_intersect_x
                l_ret_z = l_intersect_z
              end
            end
            l_within_sight, l_tangent_dist, l_tangent_x, l_tangent_z, l_goto_node = compute_tangent_dist(
in_heading, in_x, in_z, i)
            if l_within_sight and l_tangent_dist <= l_ret_intersect_dist and l_tangent_dist <=
l_max_dist_node then
              l_ret_intersect_dist = l_tangent_dist
              l_ret_segment_index = i
              l_ret_x = l_tangent_x
              l_ret_z = l_tangent_z
            end
          end
        end
```

```
    local l_decision = 0
    if l_ret_node_dist < 9999 and l_ret_intersect_dist < 9999 then
      if l_ret_intersect_dist <= l_ret_node_dist then l_decision = 1 else l_decision = 2 end
    elseif l_ret_intersect_dist < 9999 then
      l_decision = 1
    elseif l_ret_node_dist < 9999  then
      l_decision = 2
    end

    if l_decision == 0 then
      return false, 0, 0, 0, 0
    elseif l_decision == 1 then
      local l_new_node, l_new_segment = add_new_taxinode_segment (l_ret_segment_index, l_ret_x,
l_ret_z, l_ret_intersect_dist)
      return true, l_new_node, l_ret_x, l_ret_z, l_new_segment
    elseif l_decision == 2 then
      return true, l_ret_deadnode, 0, 0, 0
    end

  end

  function compute_tangent_dist( in_heading, in_x, in_z, in_idx)
    local l_angle, l_dir, l_tangent_heading = 0, 0, 0
    local l_head, l_dist = 0, 0
    local l_tangent_dist, l_tangent_x, l_tangent_z, l_goto_node = 0, 0, 0, 0
    local l_in_sight = false
    local l_node1_x = t_taxinode[t_segment[in_idx].Node1 + 1].x
    local l_node1_z = t_taxinode[t_segment[in_idx].Node1 + 1].z
    local l_node2_x = t_taxinode[t_segment[in_idx].Node2 + 1].x
    local l_node2_z = t_taxinode[t_segment[in_idx].Node2 + 1].z
    local l_seg_heading = t_segment[in_idx].Heading

    l_in_sight = chk_line_of_sight (in_heading, 60, 60, in_x, in_z, l_node2_x, l_node2_z)
    if l_in_sight then
      l_goto_node = t_segment[in_idx].Node2
    else
      l_in_sight = chk_line_of_sight (in_heading, 60, 60, in_x, in_z, l_node1_x, l_node1_z)
      if l_in_sight then
        l_goto_node = t_segment[in_idx].Node1
```

```
    else
      return false, 0, 0, 0, 0
    end
  end

  l_head, l_dist = heading_n_dist( in_x, in_z, l_node2_x, l_node2_z)
  l_angle, l_dir = compute_angle_diff( l_seg_heading , l_head )
  l_tangent_dist = math.abs( math.sin ( math.rad ( l_angle ) ) * l_dist )
  l_tangent_heading = add_delta_clockwise( l_seg_heading, 90, l_dir)
  l_tangent_x = in_x + math.sin ( math.rad ( l_tangent_heading) ) * l_tangent_dist
  l_tangent_z = in_z + math.cos ( math.rad ( l_tangent_heading) ) * l_tangent_dist * -1

  if ( ( l_tangent_x >= l_node1_x and l_tangent_x <= l_node2_x ) or
  ( l_tangent_x >= l_node2_x and l_tangent_x <= l_node1_x ) ) or
  ( ( l_tangent_z >= l_node1_z and l_tangent_z <= l_node2_z ) or
  ( l_tangent_z >= l_node2_z and l_tangent_z <= l_node1_z ) ) then
    return true, l_tangent_dist, l_tangent_x, l_tangent_z, l_goto_node
  else
    return false, 0, 0, 0, 0
  end
end

function compute_intersection (in_type, in_heading, in_x, in_z, in_idx)
  local l_node1_x = t_taxinode[t_segment[in_idx].Node1 + 1].x
  local l_node1_z = t_taxinode[t_segment[in_idx].Node1 + 1].z
  local l_node2_x = t_taxinode[t_segment[in_idx].Node2 + 1].x
  local l_node2_z = t_taxinode[t_segment[in_idx].Node2 + 1].z
  local l_seg_heading = t_segment[in_idx].Heading

  if l_seg_heading == in_heading or l_seg_heading == add_delta_clockwise( in_heading, 180, 1)
then
    return false, 0, 0, 0
  end

  local l_intersect_x, l_intersect_z = 0, 0

  if in_heading == 0 or in_heading == 180 then
    l_intersect_z =  math.tan ( math.rad ( 90 - l_seg_heading ) ) * ( in_x - l_node1_x ) +  ( l_node1_z *
-1 )
    l_intersect_z = -1 * l_intersect_z
```

```
  l_intersect_x = in_x
elseif l_seg_heading == 0 or l_seg_heading == 180 then
  l_intersect_z = math.tan ( math.rad ( 90 - in_heading ) ) * ( l_node1_x - in_x ) +  ( in_z * -1 )
  l_intersect_z = -1 * l_intersect_z
  l_intersect_x = l_node1_x
else
  local l_heading_ratio = math.tan (math.rad ( 90 - in_heading)) / math.tan (math.rad ( 90 -
l_seg_heading))
  local l_var1 =  -1 * ( l_heading_ratio * ( l_node1_z * -1 ) )
  local l_var2 =  math.tan (math.rad ( 90 - in_heading)) * (l_node1_x - in_x)
  local l_var3 = in_z * -1
  local l_var4 = 1 - l_heading_ratio
  l_intersect_z = -1 * ( ( l_var1 + l_var2 + l_var3 ) / l_var4 )
  l_var1 = -1 * ( 1 / l_heading_ratio ) * l_node1_x
  l_var2 = -1 * ( l_node1_z - in_z) / math.tan (math.rad ( 90 - in_heading))
  l_var3 = in_x
  l_var4 = 1 - ( 1 / l_heading_ratio )
  l_intersect_x = ( l_var1 + l_var2 + l_var3 ) / l_var4
end

if ( ( l_intersect_x >= l_node1_x and l_intersect_x <= l_node2_x ) or
( l_intersect_x >= l_node2_x and l_intersect_x <= l_node1_x ) ) and
( ( l_intersect_z >= l_node1_z and l_intersect_z <= l_node2_z ) or
( l_intersect_z >= l_node2_z and l_intersect_z <= l_node1_z ) ) then
else
  return false, 0, 0, 0
end

local l_heading = 0
if in_type == "gate" or in_type == "misc" then
  l_heading = add_delta_clockwise( in_heading, 180, 1)
else
  l_heading = in_heading
end

local l_in_sight, _, l_dist_to_intersect = chk_line_of_sight (l_heading, 70, 70, in_x, in_z,
l_intersect_x, l_intersect_z)

if l_in_sight == false and ( in_type == "tie_down" or in_type == "hangar" ) then
  l_heading = add_delta_clockwise( in_heading, 180, 1)
```

```
      l_in_sight, _, l_dist_to_intersect = chk_line_of_sight (l_heading, 70, 70, in_x, in_z, l_intersect_x,
l_intersect_z)
    end

    return l_in_sight, l_intersect_x, l_intersect_z, l_dist_to_intersect
  end

  function check_deadend_node (in_type, in_heading, in_x, in_z, in_idx)
    local l_deadnode = -1
    if not string.find(t_taxinode[t_segment[in_idx].Node1 + 1].Segment, ",") then
      l_deadnode = t_segment[in_idx].Node1
    elseif not string.find(t_taxinode[t_segment[in_idx].Node2 + 1].Segment, ",") then
      l_deadnode = t_segment[in_idx].Node2
    end

    if l_deadnode ~= -1 then
      local l_in_rev_heading = 0
      local l_node_x = t_taxinode[l_deadnode + 1].x
      local l_node_z = t_taxinode[l_deadnode + 1].z
      local l_heading = 0
      l_in_rev_heading = add_delta_clockwise( in_heading, 180, 1)
      if in_type == "gate" or in_type == "misc" then
        l_heading = l_in_rev_heading
      else
        l_heading = in_heading
      end
      local l_in_sight, _, l_dist_to_node = chk_line_of_sight (l_heading, 80, 80, in_x, in_z, l_node_x,
l_node_z)
      return l_in_sight, l_deadnode, l_dist_to_node
    else
      return false, 0, 0
    end
  end

  function add_new_taxinode_segment(in_segment_index, in_x, in_z, in_intersect_dist)
    local l_new_node, l_new_segment = 0, 0
    local l_idx = #t_taxinode + 1
    t_taxinode[l_idx] = {}
    t_taxinode[l_idx].x = in_x
    t_taxinode[l_idx].y = t_taxinode[t_segment[in_segment_index].Node2 + 1].y
```

```
                t_taxinode[l_idx].z = in_z
                t_taxinode[l_idx].Type = "New"
                t_taxinode[l_idx].Runway = ""
                t_taxinode[l_idx].Segment = ""
                t_taxinode[l_idx].f_value = nil; t_taxinode[l_idx].g_value = nil; t_taxinode[l_idx].h_value = nil
                t_taxinode[l_idx].parent = nil; t_taxinode[l_idx].cost = nil; t_taxinode[l_idx].heading = nil
                l_new_node = l_idx - 1

                if string.find(t_taxinode[t_segment[in_segment_index].Node1 + 1].Segment, ",") then
                  l_new_segment = #t_segment + 1
                  t_segment[l_new_segment] = {}
                  t_segment[l_new_segment].ID = "ADD_NEWSEGMENT"
                  t_segment[l_new_segment].Node1 = t_segment[in_segment_index].Node1
                  t_segment[l_new_segment].Node2 = l_new_node
                  t_segment[l_new_segment].Dir = t_segment[in_segment_index].Dir
                  t_segment[l_new_segment].Type = t_segment[in_segment_index].Type
                  t_segment[l_new_segment].Size = t_segment[in_segment_index].Size
                  t_segment[l_new_segment].Hotzone = t_segment[in_segment_index].Hotzone
                  t_segment[l_new_segment].Heading, t_segment[l_new_segment].Dist = heading_n_dist(
t_taxinode[t_segment[in_segment_index].Node1 + 1].x,
                  t_taxinode[t_segment[in_segment_index].Node1 + 1].z,
                  in_x, in_z)
                  t_taxinode[t_segment[in_segment_index].Node1+1].Segment =
t_taxinode[t_segment[in_segment_index].Node1+1].Segment..",".tostring(l_new_segment)
                  t_taxinode[l_idx].Segment = tostring(l_new_segment)
                end

                if string.find(t_taxinode[t_segment[in_segment_index].Node2 + 1].Segment, ",") then
                  l_new_segment = #t_segment + 1
                  t_segment[l_new_segment] = {}
                  t_segment[l_new_segment].ID = "ADD_NEWSEGMENT"
                  t_segment[l_new_segment].Node1 = l_new_node
                  t_segment[l_new_segment].Node2 = t_segment[in_segment_index].Node2
                  t_segment[l_new_segment].Dir = t_segment[in_segment_index].Dir
                  t_segment[l_new_segment].Type = t_segment[in_segment_index].Type
                  t_segment[l_new_segment].Size = t_segment[in_segment_index].Size
                  t_segment[l_new_segment].Hotzone = t_segment[in_segment_index].Hotzone
                  t_segment[l_new_segment].Heading, t_segment[l_new_segment].Dist = heading_n_dist( in_x,
in_z,
                  t_taxinode[t_segment[in_segment_index].Node2 + 1].x,
```

```
                t_taxinode[t_segment[in_segment_index].Node2 + 1].z)
                t_taxinode[t_segment[in_segment_index].Node2+1].Segment =
t_taxinode[t_segment[in_segment_index].Node2+1].Segment..",".tostring(l_new_segment)
                if t_taxinode[l_idx].Segment == "" then
                  t_taxinode[l_idx].Segment = tostring(l_new_segment)
                else
                  t_taxinode[l_idx].Segment = t_taxinode[l_idx].Segment..",".tostring(l_new_segment)
                end
              end
              return l_new_node, l_new_segment
            end

            function auto_assign_gate()
              local l_index = 0
              if #t_gate == 0 then arrival_gate = 0; return "-1" end

              t_suitable_gates = {}
              for l_index = 1, #t_gate  do
                if  string.match(t_gate[l_index].Types, Aircraft_Type) == Aircraft_Type or
                ( Aircraft_Type == "0" and string.match(t_gate[l_index].Types, "7" ) ) then
                  t_suitable_gates[#t_suitable_gates+1] = l_index
                end
              end
              rampstart_chg = true
              math.randomseed(os.time())
              if #t_suitable_gates > 0 then
                arrival_gate = t_suitable_gates[ math.random(1, #t_suitable_gates) ]
                gatetext = t_gate[arrival_gate].ID
                return "2"
              else
                arrival_gate = math.random(1, #t_gate)
                gatetext = t_gate[arrival_gate].ID
                if ( Err_Msg[1].text ~= nil and string.find( Err_Msg[1].text, "No suitable gate for this plane. Lift" ) )
then
                else
                  update_msg("-17")
                end
                return "1"
              end
            end
```

```
function check_gate()
  local l_dist = 0
  local l_dist_min = 9999
  local l_navid_index = 9999
  if #t_gate == 0 then return 0 end
  if fm_gnd_spd < 0.5 then
    for i = 1, #t_gate do
      _, l_dist = heading_n_dist(fm_plane_x, fm_plane_z, t_gate[i].x, t_gate[i].z)
      if l_dist < l_dist_min then
        l_dist_min = l_dist
        l_navid_index = i
      end
    end
    if l_dist_min <= 10 then
      rampstart_chg = true
      return l_navid_index
    else
      return 0
    end
  else
    return 0
  end
end

function closed_window(wnd) end

  function build_holder(wnd, x, y)
    local l_win_width = imgui.GetWindowWidth()
    local l_win_height = imgui.GetWindowHeight()
    local l_text = "FM"
    local l_text_width, l_text_height = imgui.CalcTextSize(l_text)
    imgui.SetCursorPos( l_win_width / 2 - l_text_width / 2,  l_win_height / 2 - l_text_height / 2)
    imgui.TextUnformatted(l_text)
    imgui.DrawList_AddCircle(l_win_width / 2, l_win_height / 2, 12, 0xFFFFFFFF)

    if imgui.IsMouseDragging(0) then
      if holder_drag == 0 then
        holder_drag = 1
        prev_mouse_Y = MOUSE_Y
```

```
    end
    if holder_drag > 0 then
      l_diff = (MOUSE_Y - prev_mouse_Y)
      if math.abs(l_diff) > 0 then
        Win_Y = Win_Y -  l_diff
        prev_mouse_Y = MOUSE_Y
        float_wnd_set_position(holder_wnd, screen_width - Holder_len,  Win_Y)
        holder_drag = 3
      end
    end
  end

  if imgui.IsMouseReleased(0) then
    if fm_gear1_gnd ~= 0 and fm_gear2_gnd ~= 0 then
      if holder_drag < 2 then
        toggle_window = true
      end
      holder_drag = 0
    end
  end
end

function closed_holder(wnd) end

  function load_config()
    local l_file
    local l_line = ""
    local l_str1, l_str2 = "" , ""
    local l_aircraft_type = ""

    l_file = io.open( syspath .. "Output/preferences/FollowMe.prf", "r")
    if l_file == nil then return "-2" end

    repeat
      l_line = l_file:read("*l")
      if l_line then
        l_str1, l_str2 = string.match(l_line,  "([%p%a%d]+)%S-(.*)" )
        l_str2 = trim_str (l_str2)
        if l_str1 == "holder_ypos" then
          Win_Y = tonumber( l_str2 )
```

```
        elseif l_str1 == "get_from_SimBrief" then
          get_from_SimBrief = (l_str2 == "1")
        elseif l_str1 == "simbrief_id" then
          simbrief_id = l_str2 or ""
        elseif l_str1 == "show_path" then
          show_path = (l_str2 == "1")
        elseif l_str1 == "show_rampstart" then
          rampstart_chg = true
          show_rampstart = (l_str2 == "1")
        elseif l_str1 == PLANE_ICAO then
          l_aircraft_type = l_str2
          Aircraft_Type = l_str2
        elseif l_str1 == "random_gate" then
          random_gate = (l_str2 == "1")
        elseif l_str1 == "vol" then
          vol = tonumber( l_str2 )
          set_sound_vol()
        elseif l_str1 == "speed_limiter" then
          speed_limiter = (l_str2 == "1")
        elseif l_str1 == "car_type_fmcar" then
          if l_str2 == "Ferrari" then car_type_fmcar = "Ferrari"
        elseif l_str2 == "Van" then car_type_fmcar = "Van"
        elseif l_str2 == "Truck" then car_type_fmcar = "Truck"
        elseif l_str2 == "Auto" then car_type_fmcar = "Auto"
        end
      end
    end
  until not l_line
  l_file:close()

  if l_aircraft_type ~= "" then return "" else return "-1" end
end

function save_config()
  local l_file
  local l_line = ""
  local l_content = ""
  local l_str1, l_str2 = "", ""

  l_content = l_content .. "holder_ypos" .. "t" .. tostring(Win_Y) .. "n"
```

```
  l_content = l_content .. "get_from_SimBrief" .. "t" .. (get_from_SimBrief and "1" or "") .. "n"
  l_content = l_content .. "simbrief_id" .. "t" .. simbrief_id .. "n"

  l_content = l_content .. "show_path" .. "t" .. (show_path and "1" or "") .. "n"
  l_content = l_content .. "show_rampstart" .. "t" .. (show_rampstart and "1" or "") .. "n"
  l_content = l_content .. PLANE_ICAO .. "t" .. Aircraft_Type .. "n"
  l_content = l_content .. "random_gate" .. "t" .. (random_gate and "1" or "") .. "n"
  l_content = l_content .. "vol" .. "t" .. tostring(vol) .. "n"
  l_content = l_content .. "speed_limiter" .. "t" .. (speed_limiter and "1" or "") .. "n"

  if car_type_fmcar == "Ferrari" then l_content = l_content .. "car_type_fmcartFerrarin"
elseif car_type_fmcar == "Van" then l_content = l_content .. "car_type_fmcartVann"
elseif car_type_fmcar == "Truck" then l_content = l_content .. "car_type_fmcartTruckn"
elseif car_type_fmcar == "Auto" then l_content = l_content .. "car_type_fmcartAuton"
end

l_file = io.open( syspath .. "Output/preferences/FollowMe.prf", "r")
if l_file ~= nil then
  repeat
    l_line = l_file:read("*l")
    if l_line then
      l_str1, l_str2 = string.match(l_line,  "([%p%a%d]+)%S-(.*)" )
      if l_str1 ~= "holder_ypos" and
      l_str1 ~= "get_from_SimBrief" and
      l_str1 ~= "get_from_FMS" and
      l_str1 ~= "simbrief_id" and
      l_str1 ~= "show_path" and
      l_str1 ~= PLANE_ICAO and
      l_str1 ~= "random_gate" and
      l_str1 ~= "vol" and
      l_str1 ~= "speed_limiter" and
      l_str1 ~= "car_type_fmcar" then
        l_content = l_content .. l_line .. "n"
      end
    end
  until not l_line
  l_file:close()
end
```

```
  l_file = io.open( syspath .. "Output/preferences/FollowMe.prf", "w")
  if l_file == nil then return "-4" end
  l_file:write (l_content)
  l_file:close()
  return "2"
end

function trim_str (in_str)
  local out_str = ""
  if in_str == nil then return "" end
  out_str = string.match(in_str, "%S.*")
  if out_str ~= nil then
    out_str = string.match(out_str, ".*%S")
  else
    out_str = ""
  end
  return out_str
end

function exit_plugin()
  unload_object()
  unload_probe()
  unload_path()
  unload_rampstart()
  if dataref_register1 ~= nil then XPLM.XPLMUnregisterDataAccessor(dataref_register1) end
  if dataref_register7 ~= nil then XPLM.XPLMUnregisterDataAccessor(dataref_register7) end
  if dataref_register2 ~= nil then XPLM.XPLMUnregisterDataAccessor(dataref_register2) end
  if dataref_register3 ~= nil then XPLM.XPLMUnregisterDataAccessor(dataref_register3) end
  if dataref_register4 ~= nil then XPLM.XPLMUnregisterDataAccessor(dataref_register4) end
  if dataref_register5 ~= nil then XPLM.XPLMUnregisterDataAccessor(dataref_register5) end
  XPLM.XPLMUnregisterDataAccessor(dataref_register6)
end

XPLM.XPLMGetSystemPath(char_str)
syspath = ffi.string(char_str)

register_dataref()
load_probe()

do_every_frame("handle_plugin_window()")
```

```
do_every_frame("object_physics()")
do_on_exit("exit_plugin()")
```